

# ЛЕКЦИЯ 2

ПРОДВИНУТЫЙ HTML

# ВОПРОСЫ

1. Основы HTML
2. Семантика
3. Современные подходы к структуре HTML документов
4. Формы и их расширенные возможности
5. Интеграция мультимедиа

1

# ОСНОВЫ HTML

# HTML

**(HyperText Markup Language)** — язык разметки для создания и структурирования веб-страниц. Он помогает определять, как содержимое страницы должно отображаться в браузерах.

В первую очередь, HTML — это контент и теги. Теги позволяют задать способ отображения контента на веб-страницах.





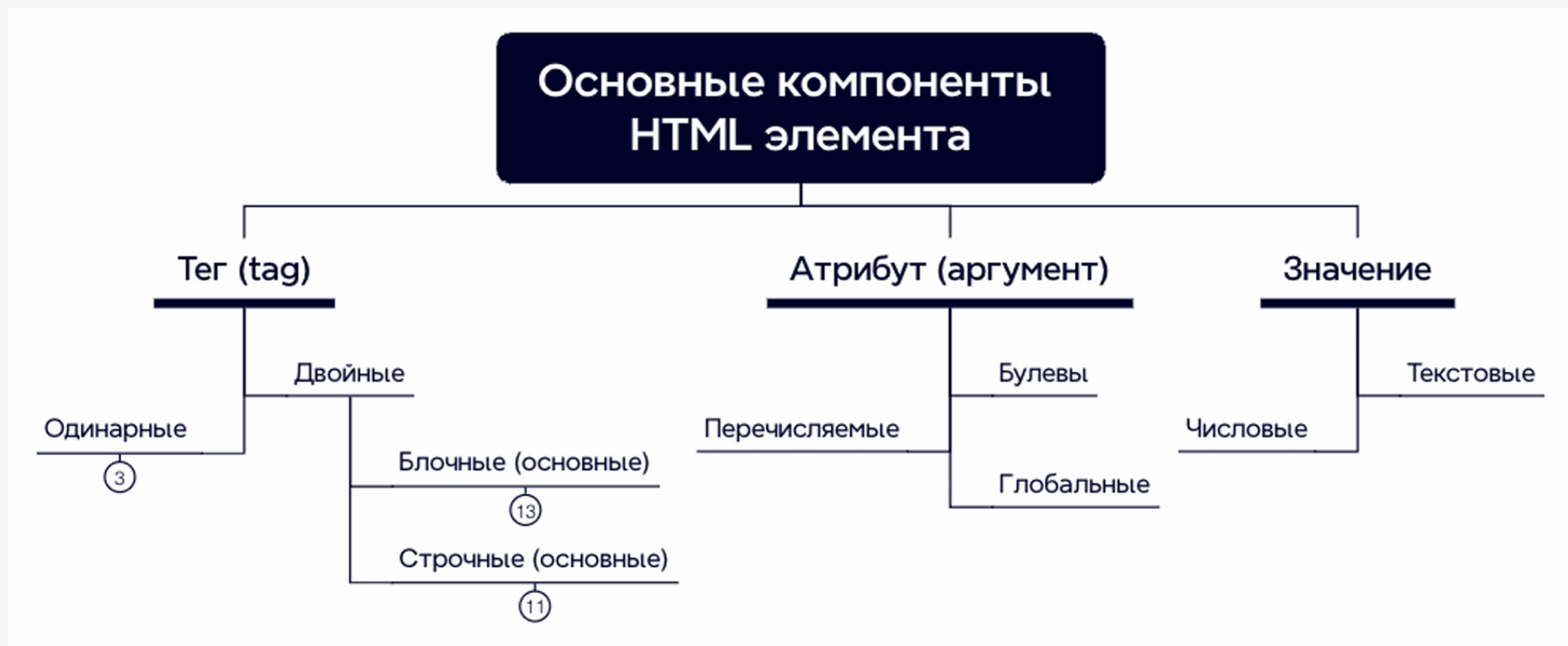
# HTML-элемент



Основными компонентами HTML элемента являются:

- **Тег (tag).** Тег HTML это компонент, который командует Web-браузеру выполнить определенную задачу типа создания абзаца или вставки изображения.
- **Атрибут (или аргумент).** Атрибут HTML изменяет тег. Например, можно выравнивать абзац или изображение внутри тега.
- **Значение.** Значения присваиваются атрибутам и определяют вносимые изменения. Например, если для тега используется атрибут выравнивания, то можно указать значение этого атрибута. Значения могут быть текстовыми, типа `left` или `right`, а также числовыми, как например ширина и высота изображения, где значения определяют размер изображения в пикселях.

# HTML-элемент



# <!DOCTYPE html>

Первым элементом любого HTML-документа является преамбула. Для HTML все, что вам нужно, это <!DOCTYPE html>. Это может выглядеть как элемент HTML, но это не так. Это специальный тип узла, называемый "doctype". Doctype указывает браузеру на использование режима стандартов. Если его не указать, браузеры будут использовать другой режим рендеринга, известный как quirks mode. Включение doctype позволяет предотвратить использование режима quirks.

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9
10 </body>
11 </html>
```

Quirks Mode - режим совместимости, используемый веб-браузерами для отображения веб-страниц, созданных до появления современных веб-стандартов. В этом режиме браузер игнорирует часть правил CSS, имитируя поведение старого движка.

# <html>

Элемент `<html>` является корневым элементом HTML-документа. Он является родителем элементов `<head>` и `<body>` и содержит все, что есть в HTML документе, кроме doctype. Если его пропустить, то он будет подразумеваться, но его важно включить, так как именно в этом элементе объявляется язык содержимого документа. Атрибут `lang` не ограничивается тегом `<html>`. Если на странице имеется текст на языке, отличном от основного языка документа, то атрибут `lang` должен использоваться для определения исключений из основного языка в документе. Атрибут `lang` не ограничивается тегом `<html>`.

```
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7    </head>
8    <body>
9
10   </body>
11 </html>
```

Если на странице имеется текст на языке, отличном от основного языка документа, то атрибут `lang` должен использоваться для определения исключений из основного языка в документе. Помимо задания языка документа и исключений из него, атрибут может использоваться в CSS:

```
<span lang="fr-fr">Ceci n'est pas</span>
```

# <head>, <meta>

Данный элемент выступает в качестве контейнера для всего содержимого, которое вы хотите включить в HTML документ, но не хотите показывать посетителям вашей страницы. Он включает такие вещи, как ключевые слова и описание страницы, которые вы хотели бы показывать в поисковых запросах, CSS для стилизования вашего контента, объявление поддерживаемого набора символов и многое другое

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9
10 </body>
11 </html>
```

<meta> в HTML нужен для указания служебной информации о странице (метаданных). С помощью метатегов можно указать: кодировку страницы; краткое описание для поисковых систем; имя автора.

# <meta>, кодировка символов

Чтобы установить кодировку символов UTF-8, включите:

```
4 <head>
5   <meta charset="UTF-8">
6   <meta property="og:title" content="Заголовок страницы Темы 2">
```

Объявив UTF-8 (без учета регистра), вы можете даже включить в заголовок символы emoji (но, пожалуйста, не надо).

Кодировка символов наследуется во всем документе, даже в <style> и <script>.

Это означает, что вы можете включать emoji в имена классов и в selectorAPI (опять же, не стоит использовать). Если вы все же используете emoji, убедитесь, что они используются таким образом, чтобы повысить удобство использования без ущерба для доступности.



# <meta>

Когда пользователи делятся ссылкой на сайт в социальных сетях, платформа автоматически формирует карточку предпросмотра. Чтобы она выглядела корректно и привлекательно, используют специальные метатеги, чаще всего по стандарту Open Graph.

```
<!DOCTYPE html>
<html lang="ru">

<head>
  <meta charset="UTF-8">
  <meta property="og:title" content="Заголовок страницы Темы 2">
  <meta property="og:description" content="Описание страницы в соцсетях. Разбираемся с тегами в теме 2">
  <meta property="og:image" content="https://m.gjcdn.net/content/750/23852121-qk5ipnag-v4.webp">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>

</body>

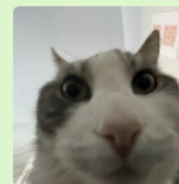
</html>
```

<https://darrmr.github.io/pr2/>

**darrmr.github.io**

**Заголовок страницы Темы 2**

Описание страницы в соцсетях. Разбираемся с тегами в теме 2



14:55 ✓✓

# <meta name="viewport">

Приведенный код означает «сделать сайт отзывчивым, начиная с того, чтобы ширина содержимого соответствовала ширине экрана».

```
8 <meta property="og:image" content="https://m.gjcdn.net/content/750/23852121-qk5ipnag-v4.webp">
9 <meta name="viewport" content="width=device-width, initial-scale=1.0">
10 <title>Страница 2</title>
```

Кроме width, можно задать масштабирование и масштабируемость, но по умолчанию они оба имеют приемлемые значения. Если вы хотите указать это явно, включите:

```
8 <meta property="og:image" content="https://m.gjcdn.net/content/750/23852121-qk5ipnag-v4.webp">
9 <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=1.0">
10 <title>Страница 2</title>
```

Viewport является частью Lighthouse accessibility audit; ваш сайт пройдет проверку, если он масштабируемый и не имеет заданного максимального размера.



# <title>

Ваша главная страница и все дополнительные страницы должны иметь уникальный заголовок.

Содержимое заголовка документа — текст между открывающим и закрывающим тегами <title> — отображается на вкладке браузера, в списке открытых окон, в истории, в результатах поиска и, если не переопределено с помощью тегов <meta> (как сделали ранее), в карточках социальных сетей

```
<> index.html > html > head > title
1  <!DOCTYPE html>
2  <html lang="ru">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta property="og:title" content="Заголовок страницы Темы 2">
7      <meta property="og:description" content="Описание страницы в соцсетях. Разбираемся с тегами в теме 2">
8      <meta property="og:image" content="https://m.gjcdn.net/content/750/23852121-qk5ipnag-v4.webp">
9      <meta name="viewport" content="width=device-width, initial-scale=1.0">
10     <title>Страница 2</title>
11     <link rel="stylesheet" href="style/style.css">
12 </head>
13
```

# <head> -> CSS

В **<head>** содержатся стили для HTML.

Существует три способа добавления CSS: **<link>**, **<style>** и атрибут **style**.

Основные два способа добавления стилей

в HTML-файл — это указание внешнего ресурса

с помощью элемента **<link>** с атрибутом **rel**, установленным на **stylesheet**, или добавление CSS непосредственно в шапку документа в открывающий и закрывающий теги **<style>**.

Тег **<link>** является предпочтительным методом добавления таблиц стилей. Ссылка на одну или несколько внешних таблиц стилей полезна как для разработчиков, так и для производительности сайта: вы получаете возможность поддерживать CSS в одном месте, а не распылять его повсюду, а браузеры могут кэшировать внешний файл, то есть его не нужно загружать заново при каждом переходе по странице.

```
11      <!--Пример 1-->
12      <link rel="stylesheet" href="style/style.css">
13      <!--Пример 2-->
14      <style>
15          * {
16              margin: 0;
17          }
18      </style>
19  </head>
20
21  <body>
22      <!--Пример 3-->
23      <header style="padding:24px;">
```

# <head> -> Favicon

```
10 <title>Много примеров</title>  
11 <link rel="stylesheet" href="style/style.css">  
12 <link rel="icon" href="assets/img/cat.png">
```

С помощью тега <link> с парой атрибутов/значений rel="icon" можно определить фавикон, который будет использоваться для вашего документа.

При большом количестве открытых вкладок вкладки уменьшаются, а заголовок может исчезнуть совсем, но значок всегда остается видимым.

Если вы не объявили фавикон, браузер будет искать файл с именем favicon.ico в каталоге верхнего уровня (корневой папке сайта). С помощью <link> можно использовать другое имя и расположение файла:

Атрибут sizes принимает значение any для масштабируемых иконок или список квадратных значений widthXheight, разделенных пробелами; если значения ширины и высоты равны 16, 32, 48 или больше в этой геометрической последовательности, единица измерения пикселей опускается, а X не зависит от регистра.



Занятия X



Много примеров

# <head> -> Скрипты

```
11 <link rel="stylesheet" href="style/style.css">  
12 <link rel="icon" href="assets/img/cat.png">  
13 <script src="script.js"></script>  
14 </head>
```

Тег <script> используется для включения скриптов.

По умолчанию используется тип JavaScript. Если вы включаете какой-либо другой язык сценариев, включите атрибут type с типом mime, или type="module", если это JavaScript-модуль.

Анализируются и выполняются только JavaScript и модули JavaScript. В JavaScript нежелательно ссылаться на элемент до того, как он будет существовать. JavaScript не только блокирует рендеринг, но и браузер останавливает загрузку всех активов при загрузке скриптов и не возобновляет загрузку других активов до тех пор, пока не будет выполнен JavaScript.

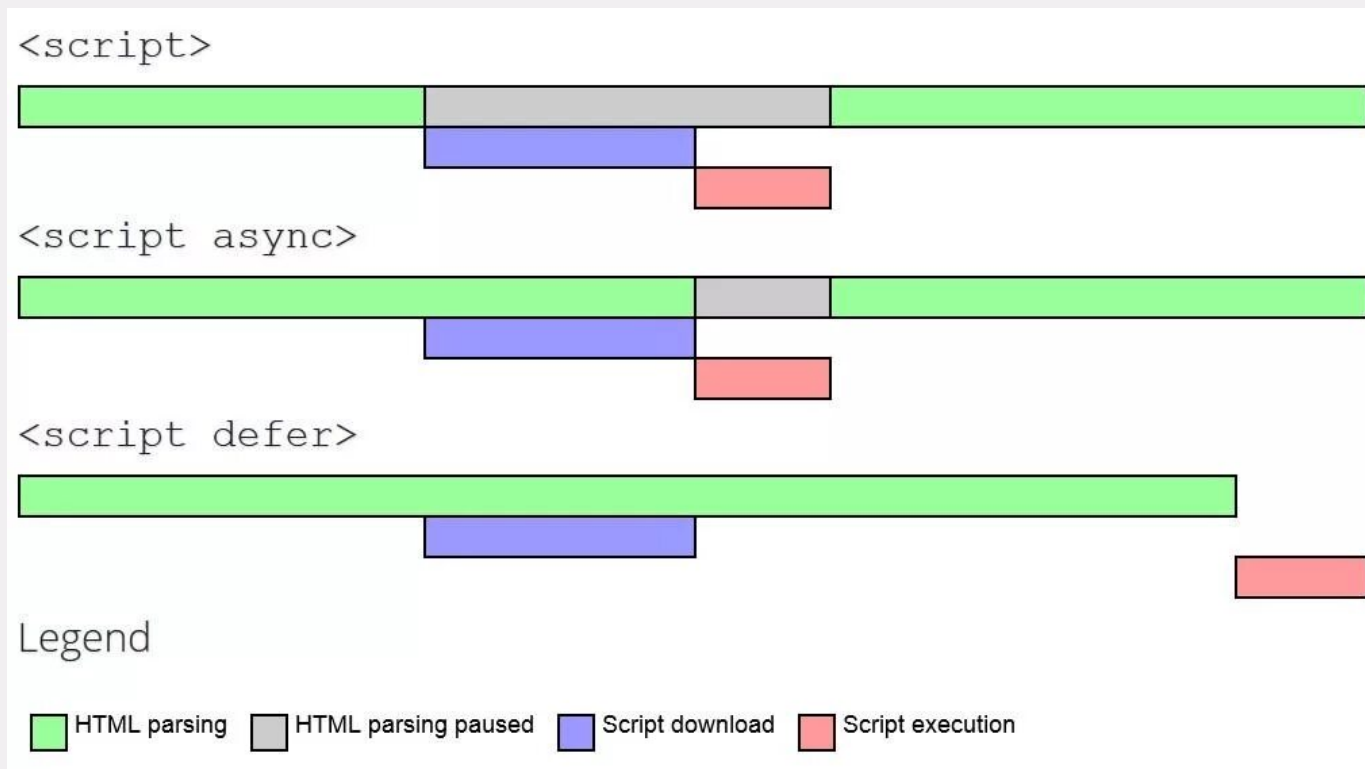
По этой причине часто можно встретить запросы JavaScript в конце документа, а не в голове

# <head> -> Скрипты

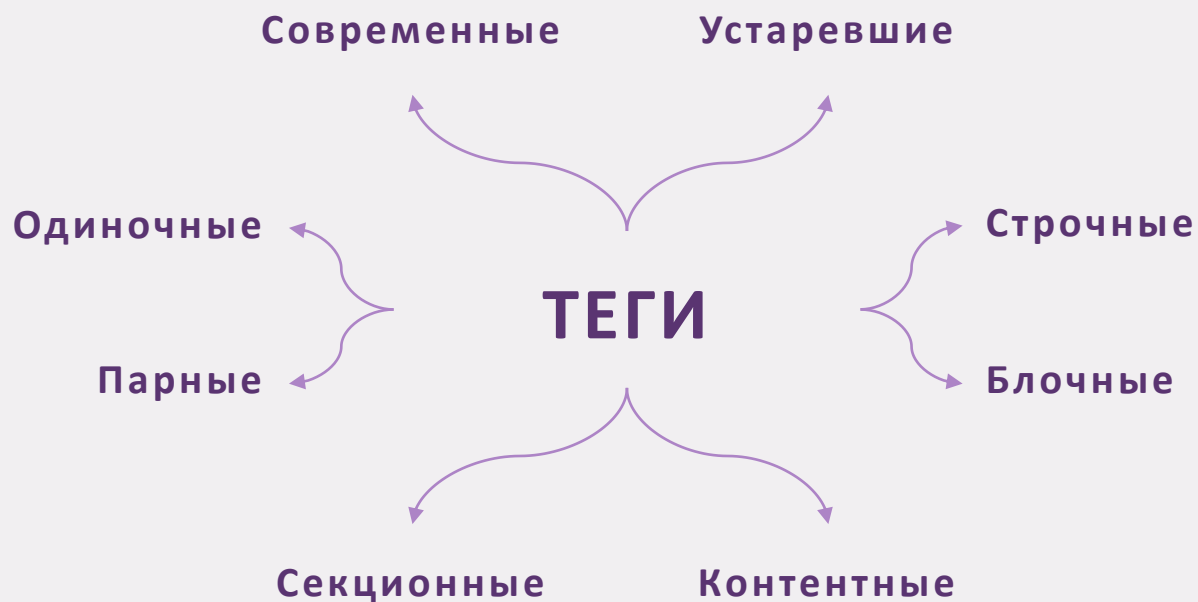
Существует два атрибута, которые могут уменьшить блокировку загрузки и выполнения JS: `defer` и `async`.

С **`defer`** рендеринг HTML не блокируется во время загрузки, а JavaScript выполняется только после завершения рендеринга документа.

С **`async`** рендеринг также не блокируется во время загрузки, но после завершения загрузки скрипта рендеринг приостанавливается на время выполнения JavaScript



# Какие вообще есть теги?



Семантические и несемантические?  
Подробнее разберемся в вопросе 2

# Секционные теги

Секционные теги делят HTML-документ (страницу) на большие логические блоки или секции:

**<main>** — основное содержимое страницы.

**<header>** — вводное содержимое или навигация. Не привязан к конкретному месту страницы и отдельной секции, но традиционно используется для создания основной навигации по сайту — шапки (хедера).

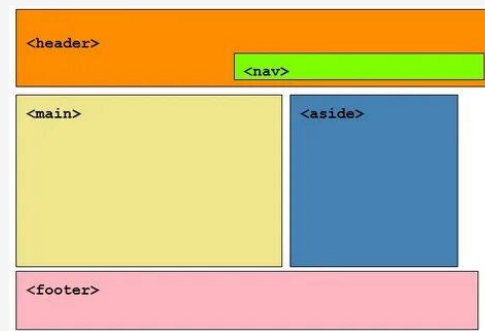
Также можно использовать для оборачивания оглавления раздела, заголовка статьи с вводной информацией о ней и прочего.

**<footer>** — дополнительная информация о сайте или части одной страницы. Тоже не привязан к части страницы или отдельной секции. Подвал (футер) может быть у превью поста и содержать информацию о дате публикации и авторе.

**<article>** — самостоятельный блок с содержимым вроде превью поста в блоге, карточки товара или рекламного баннера. Такой контент не потеряет смысл, если его перенести на другую страницу или сайт.

**<nav>** — основная или дополнительная навигация со ссылками на другие страницы или отдельные разделы.

**<section>** — блок, который объединяет содержимое по смыслу.



# Контентные теги

Контентные теги описывают смысл частичек и типов содержимого — контента.

Рассмотрим несколько наиболее часто встречающихся тегов из них:

**<h1>-<h6>** для определения заголовков на странице. **<h1>** используют для основного заголовка документа или раздела.

Остальные нужны для подзаголовков. Все заголовки описывают структуру и иерархию страниц.

**<button>** для кнопок. Используют для отправки форм и выполнения других команд.

**<img>** для картинок.

**<table>** для таблиц. Состоят из нескольких элементов для структурирования табличных данных: строк **<tr>**, заголовочных ячеек **<th>** и ячеек с данными **<td>**.

**<ul>** маркированный список.

**<ol>** нумерованный список.

**<a>** для ссылок и гиперссылок, по которым возможен переход между страницами.

**<p>** для абзацев текста. Это основные блоки текста в HTML; логически разграничивают части текста с общей мыслью или идеей.





# Periodic table of HTML elements

|         |         |            |      |      |      |      |       |        |        |          |          |          |          |          |
|---------|---------|------------|------|------|------|------|-------|--------|--------|----------|----------|----------|----------|----------|
| html    |         |            |      |      |      |      |       |        |        |          |          |          |          | output   |
| base    | header  |            |      |      |      |      |       | strong | area   | param    | caption  | th       | input    | progress |
| head    | h1-h6   |            |      |      |      |      |       | sub    | audio  | source   | col      | thead    | keygen   | select   |
| style   | hgroup  | dt         | ol   | bdi  | data | mark | ruby  | sup    | map    | canvas   | colgroup | tr       | label    | details  |
| title   | nav     | figcaption | p    | bdo  | dfn  | q    | s     | time   | track  | noscript | table    | button   | legend   | dialog   |
| body    | section | figure     | pre  | br   | em   | rp   | samp  | u      | video  | script   | tbody    | datalist | meter    | menu     |
| address | dd      | hr         | ul   | cite | i    | rt   | small | var    | embed  | del      | td       | fieldset | optgroup | menuitem |
| article | div     | li         | abbr | code | kbd  | rtc  | span  | wbr    | object | ins      | tfoot    | form     | option   | summary  |
| footer  | dl      | main       | b    |      |      |      |       |        |        |          |          |          |          |          |

|         |         |        |          |
|---------|---------|--------|----------|
| content | element | shadow | template |
|---------|---------|--------|----------|

base metadata sectioning text-content text-semantics media  
embedded script edits table forms interactive web-components

## Периодическая таблица HTML-элементов

<https://madebymike.github.io/html5-periodic-table/>

# Устаревшие теги

**Устаревшие теги и атрибуты HTML** — это элементы, которые использовались в ранних версиях HTML (например, HTML 3.2 и 4.01), но были исключены или признаны нежелательными в стандарте HTML5. Их использование сегодня не рекомендуется, так как они мешают семантической разметке, усложняют поддержку кода и нарушают принципы современного веб-дизайна.

В старых сайтах портфолио, блогах, интернет-магазинах, новостных порталах или социальных платформах можно встретить такие теги, как `<font>` для изменения цвета и размера текста или `<center>` для выравнивания содержимого.

# Примеры:

`<acronym>` — акронимы и аббревиатуры

`<blink>` — мигающий текст

`<font>` — стилизация текста

`<strike>` — зачёркнутый текст

`<bgsound>` — фоновая музыка

`<center>` — центрирование контента

`<marquee>` — бегущая строка

`<param>` — параметры для вложенных объектов

# Строчные элементы

Строчные элементы располагаются друг за другом, как слова в предложении. В зависимости от направления письма в конкретном языке элементы могут располагаться слева направо, справа налево и даже сверху вниз (как иероглифы).

Ширина и высота строчного элемента равна ширине и высоте содержимого. Несколько строчных элементов будут стремиться уместиться на одной строке, насколько хватает ширины родителя. Если ширины родителя не хватает, то лишний текст строчного элемента переносится на следующую строку.

## INLINE:



Мы не можем управлять шириной и высотой строчного элемента через CSS.

# Блочные элементы

Блочные элементы в нормальном потоке располагаются друг под другом, всегда занимая всю доступную ширину родителя. Высота блочного элемента по умолчанию равна высоте его содержимого. Три абзаца, идущие друг за другом в HTML, будут располагаться точно в таком же порядке и на странице.

Даже если ширина блочного элемента явно задана и позволяет разместить справа ещё один такой элемент, поток всё равно продолжит выстраивать их друг под другом.

## BLOCK:



# Основные теги

## STYLING: INLINE

```
<head>
<style type="text/css">
body {background-color:yellow}
p {color:blue} </style>
</head>
```

• it can also be done for a **single occurrence** (ex: paragraph):

```
<p style="color:blue;margin-left:20px;">
This is a paragraph.</p>
```

## BACKGROUND COLOR

```
<body style="background-color:yellow;">
```

• this changes the color for the entire body

```
<body style="color:yellow;">
```

• this changes just the text color for the entire body

```
<h2 style="background-color:red;">This is a heading</h2>
```

• this changes the bkgrd for the heading

```
<p style="background-color:green;">This is a paragraph.</p>
```

• this changes the bkgrd for the paragraph

## FONT, COLOR, & SIZE

```
<body>
<h1 style="font-family:verdana;">A heading</h1>
<p style="font-family:arial;color:red;font-size:20px;">A paragraph.</p>
</body>
```

## TEXT ALIGNMENT

```
<body>
<h1 style="text-align:center;">Center-aligned heading</h1>
<p>This is a paragraph.</p>
</body>
```

# HTML

## A BEGINNER'S CHEAT SHEET

## NOTEPAD TO CODE

1. Type your HTML code into your Notepad
2. Select Save as.. in Notepad's file menu with the extension .htm or .html
3. Start your web browser and open your html file from the File, Open menu, or just browse the folder and double-click your HTML file.

### BASIC SET UP

```
<!DOCTYPE html>
<html>
<body>

</body>
</html>
```



### ADD LINK

```
<a href="url">Link text</a>
```

### ADD IMAGE

```
<img src=
"http://randompicture.gif"
width="220" height="142">
```

```
<div id="container"
style="width:500px">
```

```
<div id="header"
style="background-
color:#FFA500;">
<h1 style="margin-
bottom:0;">Main Title
of Web Page</h1></div>
```

```
<div id="menu" style="background-
color:#FFD700;height:200px;width:100px;float:left;">
<b>Menu</b><br> HTML<br> CSS<br> JavaScript</div>
```

```
<div id="content" style="background-
color:#EEEEEE;height:200px;width:400px;float:left;">
Content goes here
</div>
</div>
```

#### MAIN TITLE OF WEB PAGE

| MENU | CONTENT GOES HERE |
|------|-------------------|
| HTML |                   |
| CSS  |                   |
| JAVA |                   |

## DIVIDING AREAS



```
<p>paragraph</p>
<h1>header</h1>
<br>line break
<hr> horizontal line
```

## PARAGRAPHS

## FORMAT TYPE

```
<b></b>          Defines bold text
<i></i>          italic
<small></small>   smaller text
<sub></sub>       subscripted text
<sup></sup>       superscripted text
```

Color HEX  
#000000  
#FF0000

Color RGB  
rgb(0,0,0)  
rgb(255,0,0) -red

## ENTITIES

non-breaking space &#160;  
< less than &#60;  
> greater than &#62;

## COLORS

## CONVERT HTML TO XHTML

Add an XHTML <!DOCTYPE> to the first line of every page  
Add an xmlns attribute to the html element of every page  
Change all element names to lowercase  
Close all empty elements  
Change all attribute names to lowercase  
Quote all attribute values

## COMMENTS

```
<!-- This is a comment -->
```

Text Fields: First Name

## FORMS

```
<form> First name: <input type="text" name="firstname"><br>
Last name: <input type="text" name="lastname"> </form>
```

Radio Buttons: ☐ Male ☐ Female

```
<form> <input type="radio" name="sex" value="male">Male<br>
<input type="radio" name="sex" value="female">Female </form>
```

Check Box: ☐ I have a bike ☐ I have a car

```
<form> <input type="checkbox" name="vehicle" value="Bike">I
have a bike<br>
<input type="checkbox" name="vehicle" value="Car">I have a
car </form>
```

Submit buttons: username:

```
<form name="input" action="html_form_action.asp"
method="get">
Username: <input type="text" name="user"><input
type="submit" value="Submit"></form>
```

## LISTS

- 1.
- 2.
- 3.

### UNORDERED

```
<ul>
<li>Coffee</li>
<li>Milk</li>
</ul>
```

### ORDERED

```
<ol>
<li>Coffee</li>
<li>Milk</li>
</ol>
```

## TABLES

```
<table border="1">
<tr>
<th>Header 1</th>
<th>Header 2</th>
</tr> <tr>
<td>row 1, cell 1</td>
<td>row 1, cell 2</td>
</tr> <tr>
<td>row 2, cell 1</td>
<td>row 2, cell 2</td>
</tr>
</table>
```

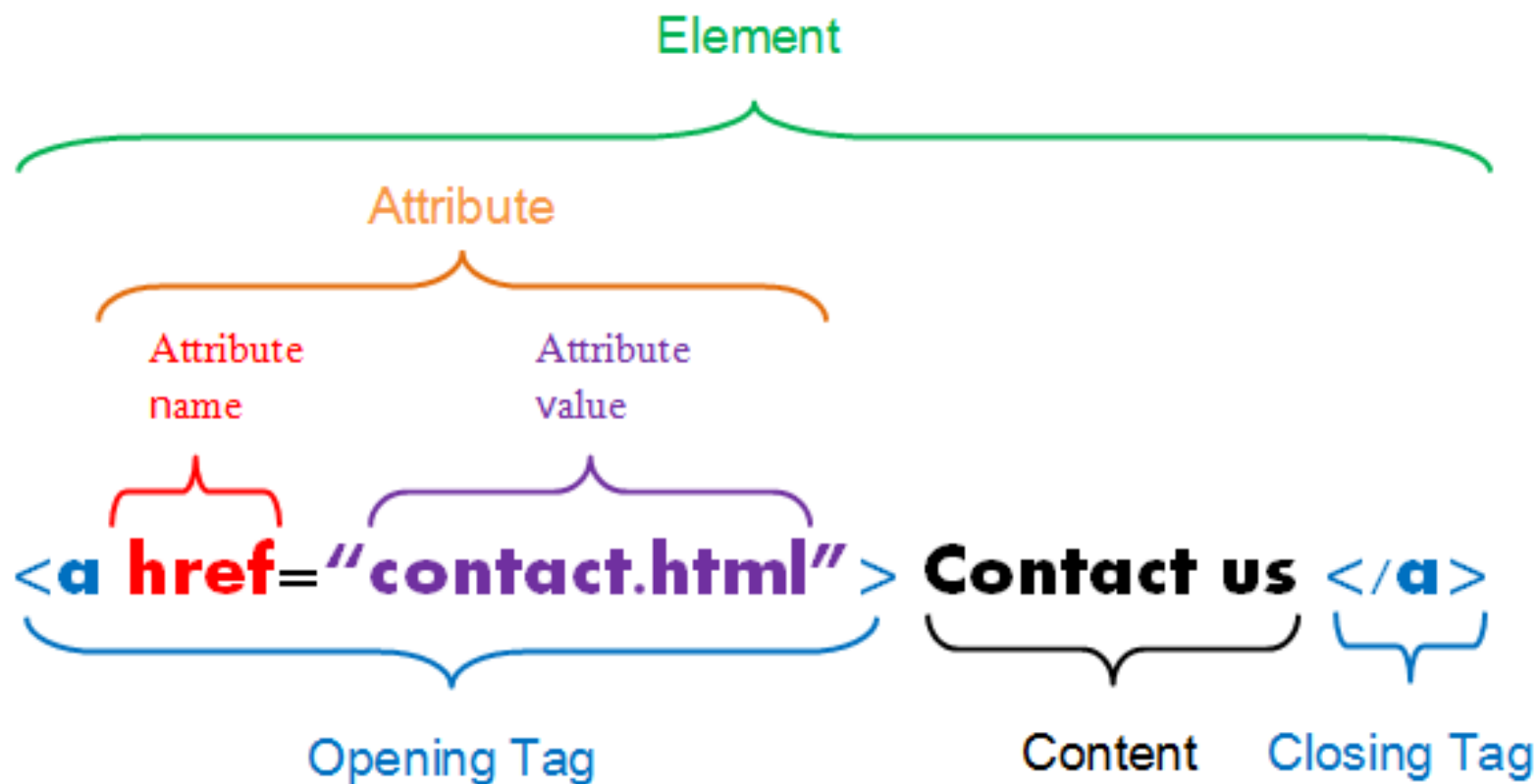
| HEADER 1      | HEADER 2      |
|---------------|---------------|
| ROW 1, CELL 1 | ROW 1, CELL 2 |
| ROW 2, CELL 1 | ROW 2, CELL 2 |

# Атрибуты тегов и их применение

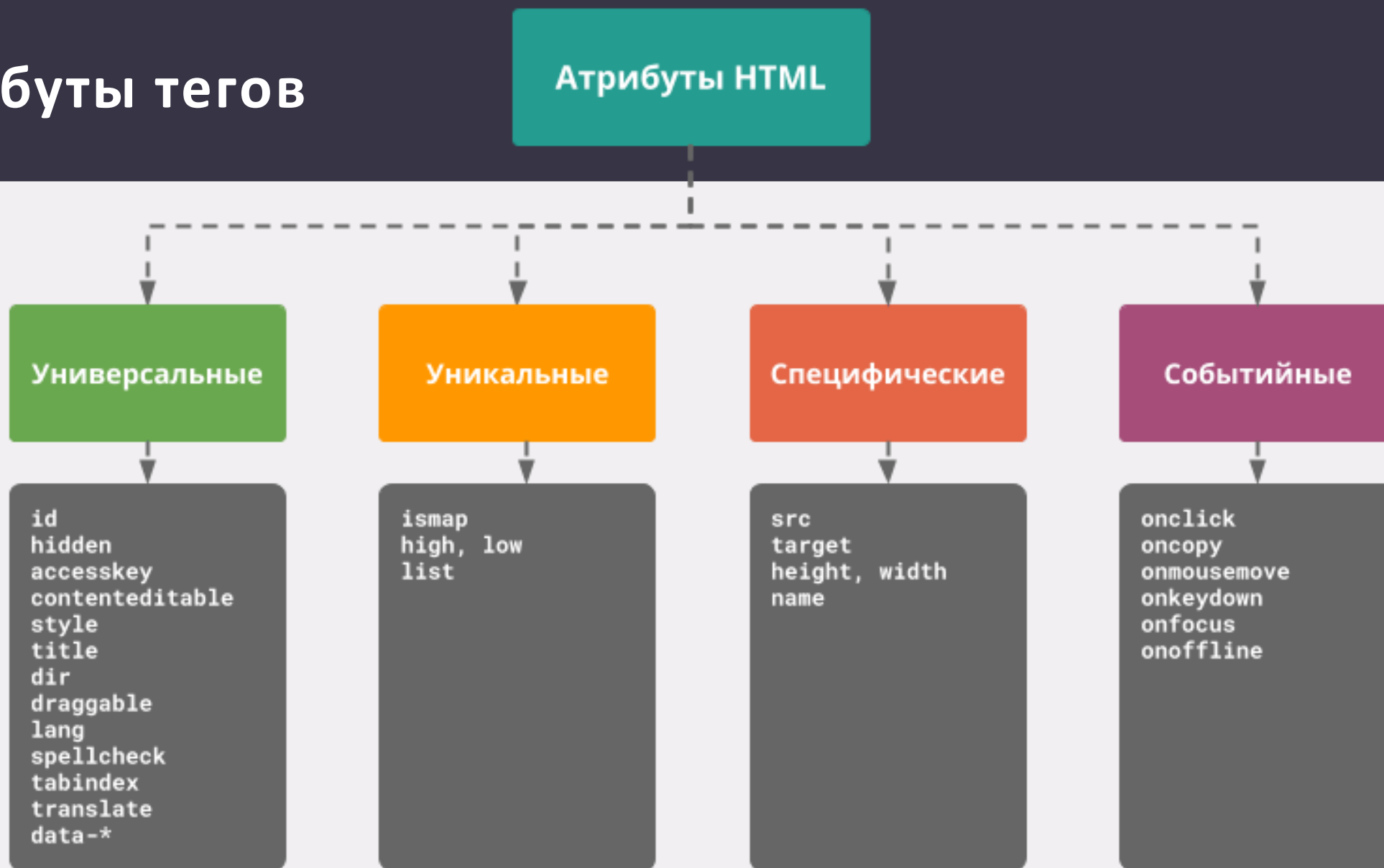
**Атрибуты в HTML** - это дополнительные параметры, представляющие информацию о элементах и влияют на их поведение и отображение на веб-странице.

Они позволяют задавать различные свойства.

Понимание работы с атрибутами является основой для создания эффективных и семантически правильных веб-страниц.



# Атрибуты тегов



# Глобальные (универсальные) атрибуты

## Атрибуты, которые подходят к каждому тегу.

```
15 <div id="uniqueElement" class="highlight" style="padding: 10px; border: 1px solid black;"  
    title="Это уникальный элемент" data-info="Дополнительные данные">  
16     Наведите курсор на этот элемент для получения дополнительной информации.  
17 </div>  
18  
19 <script>  
20     // Пример использования пользовательских данных  
21     const element = document.getElementById('uniqueElement');  
22     console.log(element.getAttribute('data-info')); // Выводит: Дополнительные данные  
23 </script>
```

- **class** определяет один или более классов для элемента, что позволяет применять стили CSS.
- **id** уникальный идентификатор элемента на странице.
- **style** встраиваемые CSS-стили для конкретного элемента.
- **title** предоставляет дополнительную информацию о элементе, которая отображается при наведении курсора.
- **data-\*** позволяет хранить пользовательские данные, которые могут быть использованы JS



# Глобальные (универсальные) атрибуты

## Глобальный атрибут id

- 1. Целевой идентификатор фрагмента ссылки.** Атрибут id позволяет создать якорные ссылки на определенные фрагменты страницы.
- 2. Идентификация элемента для сценариев.** JavaScript и другие сценарные языки могут использовать id для доступа к конкретным элементам.
- 3. Связывание элемента формы с его меткой.** В формах атрибут id используется для связывания меток (<label>) с соответствующими полями ввода.
- 4. Предоставление метки или описания для вспомогательных технологий.**
- 5. Адресация стилей в CSS.** В CSS атрибут id может быть использован для применения стилей к конкретным элементам с высокой специфичностью. Селекторы с использованием id (например, #myElement) имеют более высокий приоритет по сравнению с классами и тегами, что позволяет точно настраивать внешний вид элементов

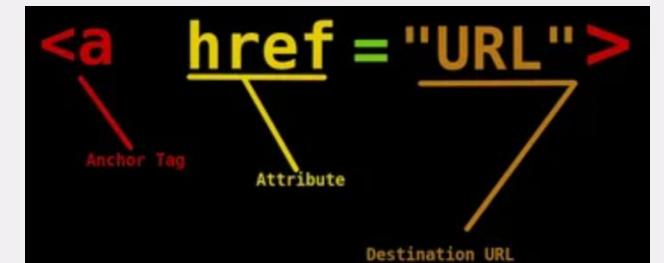
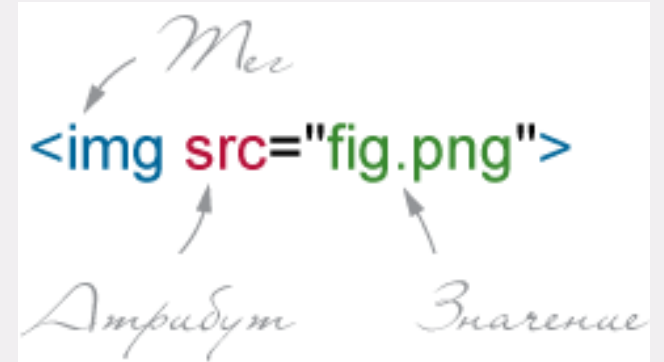


# Специфические атрибуты

Эти атрибуты применяются только к определённым элементам.

Например:

- **src** (для <img>, <script>) - указывает путь к ресурсу (изображению или скрипту).
- **href** (для <a>) - определяет URL для перехода по ссылке.
- **alt** (для <img>) - альтернативный текст для изображения, который отображается, если изображение не загружается.
- **value** (для <input>) - устанавливает значение для элементов формы.



# Перечислимые атрибуты

Эти атрибуты могут принимать только определенные значения.  
Например:

- **disabled** - если присутствует, элемент формы будет недоступен для взаимодействия.
- **checked** (для `<input type="checkbox">` или `<input type="radio">`) - указывает, что элемент выбран по умолчанию.
- **readonly** - делает элемент формы доступным только для чтения.
- **required** - указывает, что поле ввода обязательно для заполнения перед отправкой формы

## Input element should be disabled in HTML

Disabled Input Field:

Input Field

### Checkboxes

- ☐ Unchecked
- ☒ Checked
- ☐ Disabled unchecked
- ☒ Disabled checked

### Radio Buttons

- ☐ Radio is off
- ☒ Radio is on
- ☐ Disabled radio is off
- ☒ Disabled radio is on

### Switches

- ☒ ON
- ☐ OFF
- ☒ ON
- ☒ ON

# Булевы атрибуты

Булевы атрибуты (например, `required`, `disabled`, `checked`) всегда считаются истинными, если они присутствуют в элементе. Для них можно опустить значение.

```
1 <input required />
2 <input required="" />
3 <input required="required" />
```

Все значения, включая `true`, `false` и 😊, даже если они недействительны, примут значение `true`.

Если значение атрибута равно `false`, опустите атрибут. Если значение атрибута истинно, включите атрибут, но не указывайте его значение.

При переключении между `true` и `false` добавляйте и удаляйте атрибут с помощью JavaScript, а не переключая его значение

```
1 const myMedia = document.getElementById('mediaFile');
2 myMedia.removeAttribute('muted');
3 myMedia.setAttribute('muted');
```

## Пересечение между перечисляемыми и булевыми атрибутами

1. Веб-разработчики могут использовать один и тот же атрибут для управления состоянием элемента без необходимости указывать значение.

2. Это упрощает синтаксис и делает код более читаемым.

В HTML есть пересечение между булевыми и перечисляемыми атрибутами, и это может вызывать путаницу.

Некоторые атрибуты можно считать как булевыми, так и перечисляемыми, поскольку их поведение зависит от наличия или отсутствия значения. Например, атрибут **checked** у элемента `<input type="checkbox">` указывает, что чекбокс выбран, если он присутствует, и не выбран, если отсутствует.

Почему так?

Например, вместо того чтобы писать `<input type="checkbox" value="true">`, достаточно просто указать `<input type="checkbox" checked>`

# Правила использования атрибутов

## Синтаксис

Атрибуты добавляются в открывающий тег элемента в формате имя="значение". Если значение содержит пробелы или специальные символы, его следует заключать в кавычки.

## Чувствительность к регистру

HTML не чувствителен к регистру, однако значения некоторых атрибутов могут быть чувствительными.

Например, значения для `id` и `class` следует использовать с осторожностью.

```
HTML
1 <div class="my class with spaces">Содержимое</div>
2 <input type="text" value="Пример с пробелами и & специальными символами">
3
4 <div id="MyID">Первый элемент</div>
5 <div id="myid">Второй элемент</div>
6
7 <!-- В этом случае MyID и myid будут считаться разными значениями, так как id чувствителен к
   регистру. -->
8
9 <!-- Также для атрибута class: -->
10
11 <div class="myClass">Элемент с классом myClass</div>
12 <div class="MyClass">Элемент с классом MyClass</div>
```

# Пример использования атрибутов

## 1 Создание ссылки с атрибутами

```
<a href="https://example.com" target="_blank" rel="noopener noreferrer"
    title="Перейти на Example">Перейти на Example</a>
```

Здесь мы используем `href`, чтобы указать URL, `target="_blank"` для открытия ссылки в новой вкладке и `rel` для повышения безопасности. Атрибут `rel` в HTML используется для указания отношения между текущим документом и ресурсом, на который ссылается ссылка.

**noopener.** Предотвращает открытие новой вкладки или окна с доступом к объекту `window.opener`. Это повышает безопасность, так как предотвращает возможность новой страницы манипулировать исходной страницей через JavaScript.

**noreferrer.** Этот параметр не только предотвращает доступ к `window.opener`, но и не передает информацию о реферере (URL текущей страницы) на новую страницу

# Пример использования атрибутов

## 1 Форма с различными атрибутами:

В этом примере мы используем атрибуты **action** и **method** для определения того, куда отправляется форма и каким образом

Имя:  Email:

 HTML

```
1 <form action="/submit" method="POST">
2   <label for="name">Имя:</label>
3   <input type="text" id="name" name="name" required>
4   <label for="email">Email:</label>
5   <input type="email" id="email" name="email" required>
6   <button type="submit">Отправить</button>
7 </form>
```

# Специальные случаи

1. Атрибуты данных (data-\*) - позволяют хранить пользовательские данные прямо в HTML.

```
<div data-user-id="12345" data-role="admin">Пользователь</div>
```

Эти данные могут быть использованы JavaScript для динамического взаимодействия с элементами страницы.

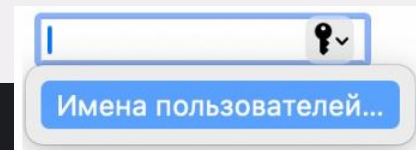
2. Стилизация с помощью встроенных стилей (могут быть полезны для быстрого тестирования, но лучше использовать внешние или внутренние таблицы стилей).

```
<div style="color: red; background-color: yellow;">Это стильный блок!</div>
```

Это стильный блок!

3. Атрибуты для форм. Формы имеют множество специфических атрибутов, таких как autocomplete, который может улучшить пользовательский опыт

```
<input type="text" name="username" autocomplete="username">
```





2

# СЕМАНТИКА

|         |         |            |     |      |     |    |       |      |       |          |       |          |          |          |
|---------|---------|------------|-----|------|-----|----|-------|------|-------|----------|-------|----------|----------|----------|
| title   | nav     | figcaption | p   | bdo  | dfn | q  | s     | time | track | noscript | table | button   | legend   | dialog   |
| body    | section | figure     | pre | br   | em  | rp | samp  | u    | video | script   | tbody | datalist | meter    | menu     |
| address | dd      | hr         | ul  | cite | i   | rt | small | var  | embed | del      | td    | fieldset | optgroup | menuitem |

**Одна из важных особенностей HTML — семантика**, которая буквально переводится как «обозначающий». При помощи HTML мы описываем структуру страниц и их содержимого, то есть объясняем браузерам, вспомогательным и другим технологиям и программам значение элементов. Это может быть параграф текста, кнопка, часть страницы с основным содержимым или ничего не значащий контейнер.

На практике семантика HTML-элементов передаётся через их роли, а ещё состояния и свойства. Роли описывают основной смысл элемента, а состояния и свойства добавляют больше деталей: выбран или не выбран чекбокс, активно или неактивно поле в форме и т.д.

## Что такое семантика?

Как работает семантика в HTML и зачем она вообще нужна.

|         |         |            |     |      |     |    |       |      |       |          |       |          |          |          |
|---------|---------|------------|-----|------|-----|----|-------|------|-------|----------|-------|----------|----------|----------|
| title   | nav     | figcaption | p   | bdo  | dfn | q  | s     | time | track | noscript | table | button   | legend   | dialog   |
| body    | section | figure     | pre | br   | em  | rp | samp  | u    | video | script   | tbody | datalist | meter    | menu     |
| address | dd      | hr         | ul  | cite | i   | rt | small | var  | embed | del      | td    | fieldset | optgroup | menuitem |

Каждый раз, когда пишете HTML-код, вы имеете дело с семантикой. Даже если используете только **<div>** и **<span>**. Все HTML-теги, которых больше 100, семантические. Для простоты ранее разделили их на две группы — секционные и контентные.

Секционные теги делят HTML-документ (страницу) на большие логические блоки или секции: **<article>**, **<nav>**, **<section>**, **<main>**, **<header>**, **<footer>**.

Контентные теги описывают смысл частичек и типов содержимого — контента.

К этим тегам относятся, например:

**<h1>-<h6>**, **<button>**, **<img>**, **<table>**, **<ul>**, **<a>**, **<p>**

## Что такое семантика?

Полный список HTML тегов: <https://html.spec.whatwg.org/multipage/>

# Что такое семантика?

HTML-атрибуты раскрывают дополнительный смысл содержимого страниц. Например, глобальный атрибут `title` означает, что это дополнительные сведения о тексте или элементе. `lang` сообщает о языке слова, фразы или ссылки, а `dir` описывает направление письма.

**Generic** — базовая роль из стандарта WAI-ARIA

Не все теги подробно описывают типы содержимого. Например, `<section>` только намекает, что это что-то объединённое по смыслу. `<article>` уже больше рассказывает о содержимом: оно тесно связано, сгруппировано в одном месте и не зависит от другого на странице.

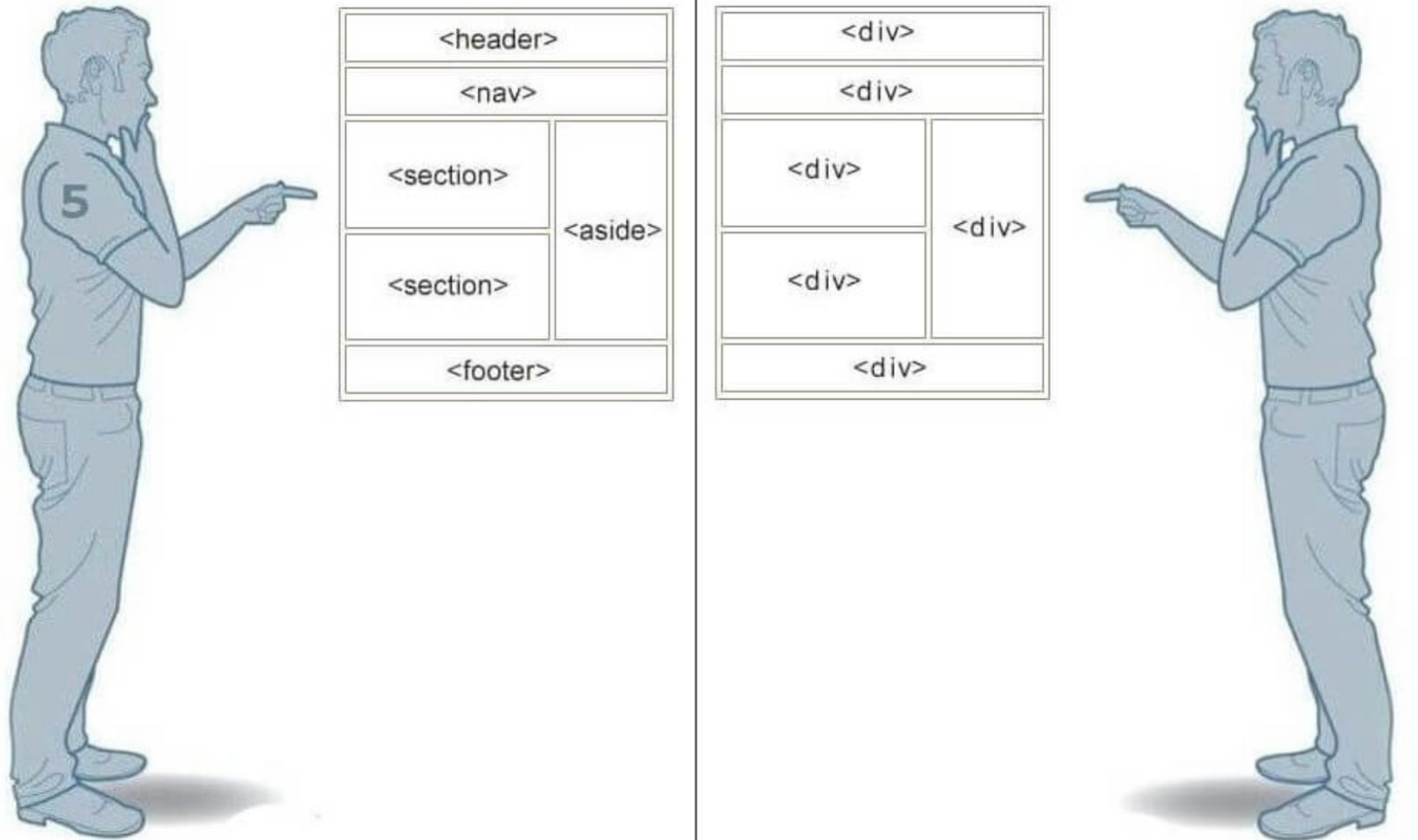
`<div>` и `<span>` тоже существуют не зря, хотя их часто называют несемантическими. Это не совсем так. Оба тега играют роль универсальных контейнеров с ролью `generic`. То есть они говорят браузерам или скринридерам, что их смысл не важен и на них можно не обращать внимание.

**Generic** не добавляет элементу специального семантического значения. Таким элементам можно задавать все глобальные ARIA-атрибуты

# Зачем нужна семантика?

Можно подумать, что раз даже `<div>` и `<span>` семантические, расставлять теги кнопок, ссылок и списков долго и бессмысленно.

Всё прямо наоборот: у вёрстки, которая подробно описывает содержимое страниц, больше преимуществ, чем у «дивной».



# 1

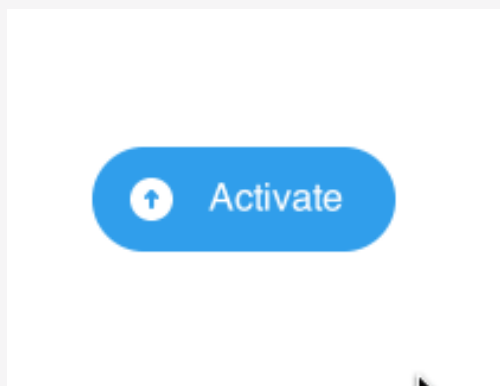
## Поддержка и читаемость кода

Исходный код проекта, построенный с вниманием к семантике, значительно легче поддерживать.

Такой код проще читать и понимать, что позволяет новым членам команды быстрее вникать в логику проекта.

Качественная вёрстка экономит время разработчиков и уменьшает количество строчек кода.

Например, у `<button>` и `<input>` уже есть стили по умолчанию и встроены обработчики событий: их не нужно писать руками. Чтобы воспроизвести нативное поведение кнопки, нужно слушать события `keydown` при нажатии на Enter и `keyup` для пробела и Enter.



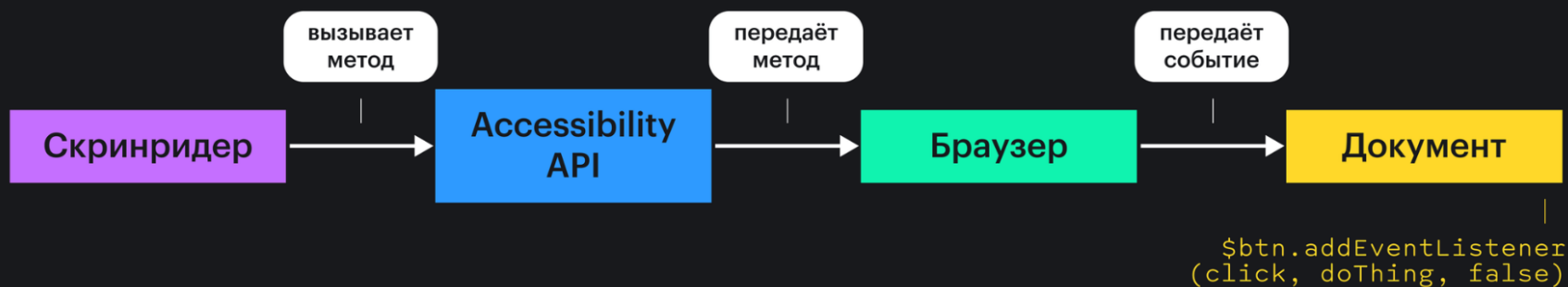
Размеры файлов со стилями и скриптами напрямую влияют на производительность сайта (performance).

Семантика является основой для хорошего пользовательского опыта (UX), особенно для людей с ограниченными возможностями.

Пользователи скринридеров полагаются на семантические теги, которые правильно озвучивают роли элементов — кнопка, заголовок, чекбокс, что позволяет понимать структуру страницы на слух.

Элементы-ориентиры, такие как `<header>` и `<footer>`, и правильная иерархия заголовков позволяют легко перемещаться по сайту с помощью клавиатуры и горячих клавиш.

Другой плюс продуманной HTML-разметки — навигация для скринридеров. Теги вроде `<header>` и `<footer>` — ориентиры. Это значит, что пользователи могут перемещаться по отдельным блокам страницы с помощью горячих клавиш.



`<header>`, `<footer>`, `<section>` и другие элементы становятся доступными для навигации, когда вложены прямо в `<body>`.

**Bionic Reading is a new method facilitating the reading process by guiding the eyes through text with artificial fixation points. As a result, the reader is only focusing on the highlighted initial letters and lets the brain center complete the word. In a digital world dominated by shallow forms of reading, Bionic Reading aims to encourage a more in-depth reading and understanding of written content.**

К примеру, во все браузеры встроен режим чтения (reading mode). В нём со страницы удаляются все лишние элементы и остаётся самое важное: заголовки, текст, картинки и видео. Если страница, например, свёрстана без `<h1>`, `<h2>` и заголовков других уровней, пользователи получат большой кусок неструктурированного текста.

Часть пользователей устанавливает разные браузерные расширения. К примеру, чтобы изменять стили страниц, текста или для бионического чтения (пример слева).

Голосовые помощники выполняют команды быстрее и точнее, если для ссылок используете `<a>` или `<time>` для дат.

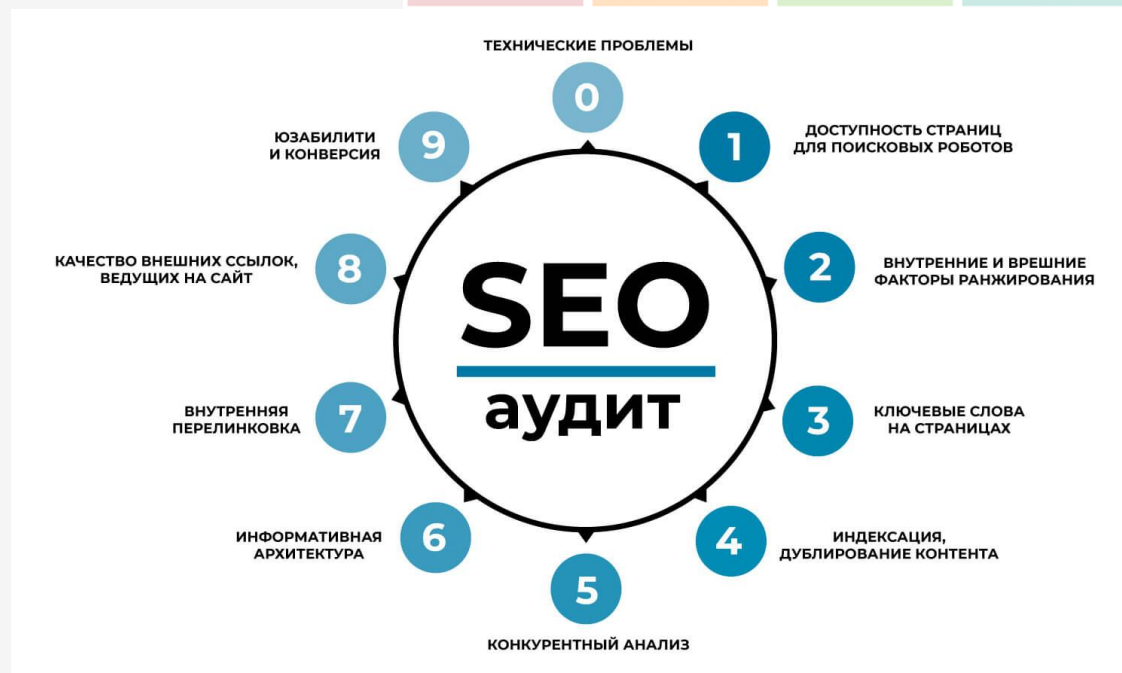


Для поисковиков семантическая верстка = четкий сигнал о структуре и содержании страницы.

Она помогает определять тему контента и его важные разделы, что положительно сказывается на индексации и ранжировании.

Чистый семантический код способствует лучшей производительности сайта, что является одним из ключевых факторов SEO.

Оценка складывается из E-E-A-T (Google): опыта (experience), экспертизы (expertise), авторитетности (authoritativeness) и надёжности (trustworthiness) контента.



Важные факторы ранжирования — микроразметка (microdata) и качество содержимого сайтов

## Зачем нужна семантическая разметка?

### Читаемость кода

Код с использованием семантических элементов легче читать и поддерживать. Это упрощает работу для других разработчиков и позволяет быстрее ориентироваться в структуре документа. Когда разработчики используют семантические элементы, они создают код, который является более интуитивно понятным и легким для понимания.

### Соответствие стандартам

Использование семантической разметки соответствует современным стандартам веб-разработки. Это делает ваш код более устойчивым к изменениям в браузерах и технологиях. Следуя стандартам, разработчики могут быть уверены в том, что их сайт будет работать корректно на большинстве устройств и платформ.

### Улучшение доступности

Одним из самых важных аспектов семантической разметки является улучшение доступности контента для людей с ограниченными возможностями. Вспомогательные технологии, такие как экранные читалки, могут распознавать семантические элементы и предоставлять пользователям более понятный контент. Например, экранная читалка может сообщить пользователю о том, что он находится в разделе навигации или что он читает статью.

### Поисковая оптимизация (SEO)

Поисковые системы используют семантику для оценки содержания страницы. Правильное использование семантических тегов может значительно улучшить видимость сайта в результатах поиска. Семантические элементы помогают поисковым системам лучше понимать структуру и содержание страницы, что может привести к более высокому ранжированию.

# Семантическая разметка

3

# СОВРЕМЕННЫЕ ПОДХОДЫ К СТРУКТУРЕ HTML ДОКУМЕНТОВ

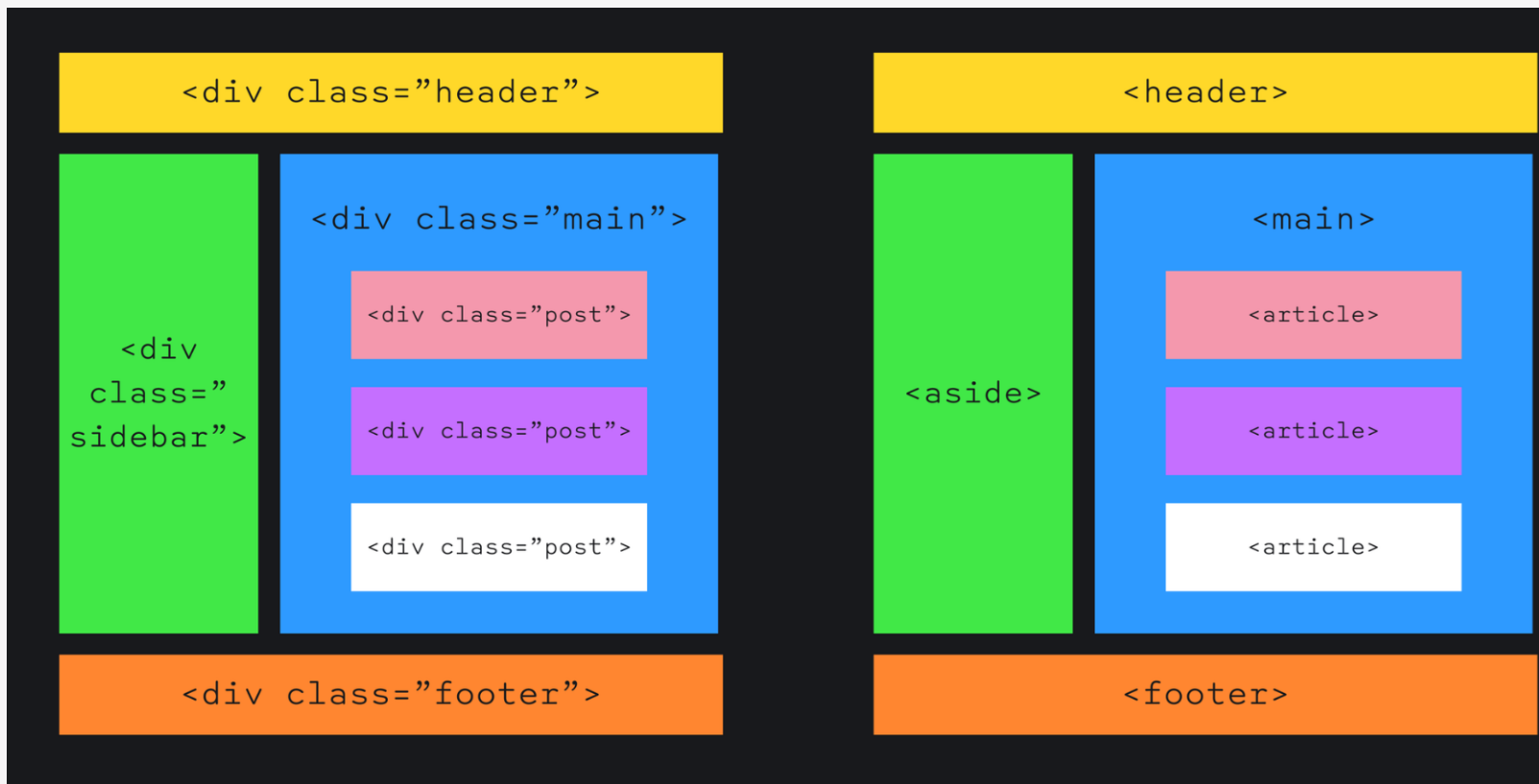
# Учитываем семантику

## 1

## Как разметить страницу

Смотрим на крупные логические блоки страницы и выбираем подходящие теги.

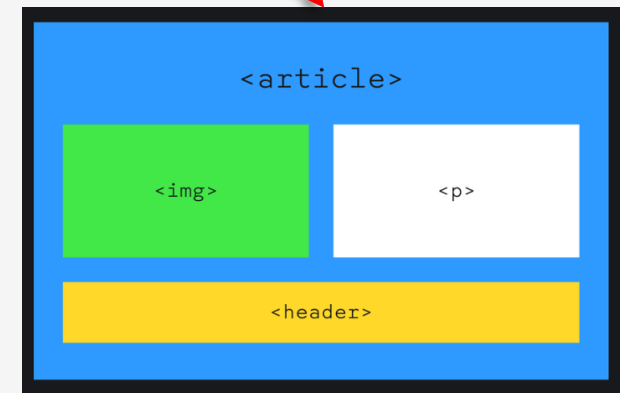
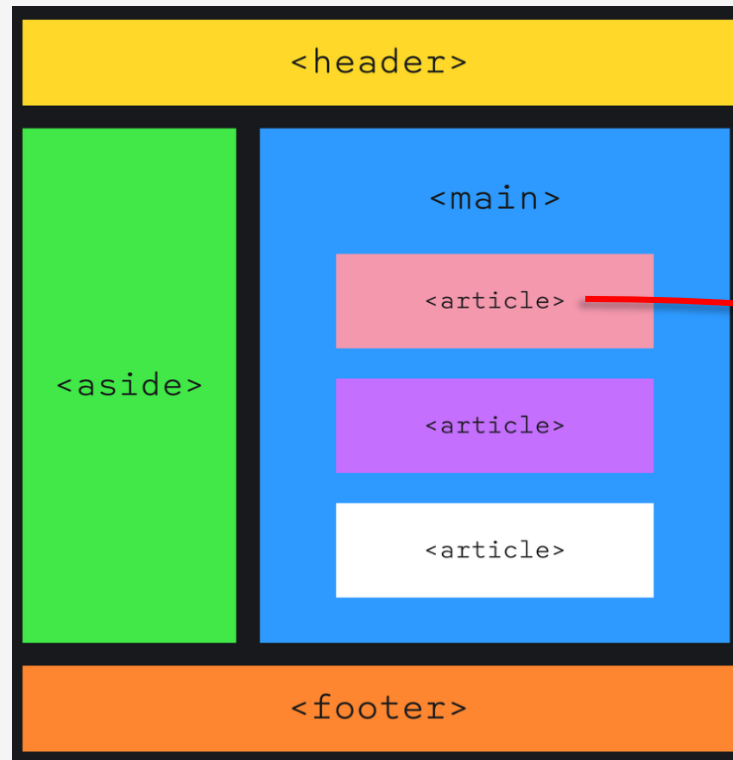
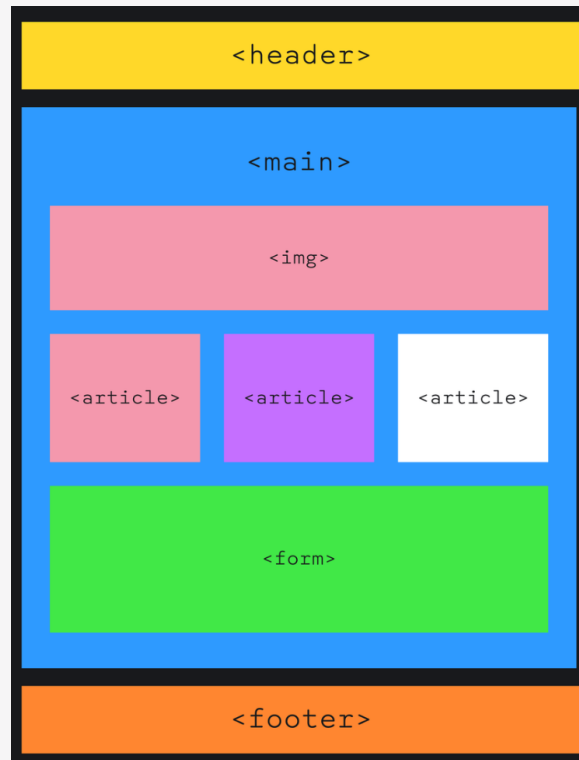
Например, `<header>`, `<footer>`, `<main>`, `<nav>` или `<aside>`.



# Учитываем семантику

1

## Как разметить страницу



## Как разметить страницу

Смотрим на содержимое всей страницы и отдельных, более мелких блоков на ней.

Расставляем заголовки `<h1>-<h6>`.

Хороший тон использовать один `<h1>`.

После в правильном порядке располагаем заголовки других уровней.

Представим, что на странице есть заголовок первого уровня и несколько подзаголовков второго и третьего уровней.

После `<h1>` идёт `<h2>`, после — пара `<h3>`, которые находятся в том же блоке. Следующая секция опять начинается с `<h2>`



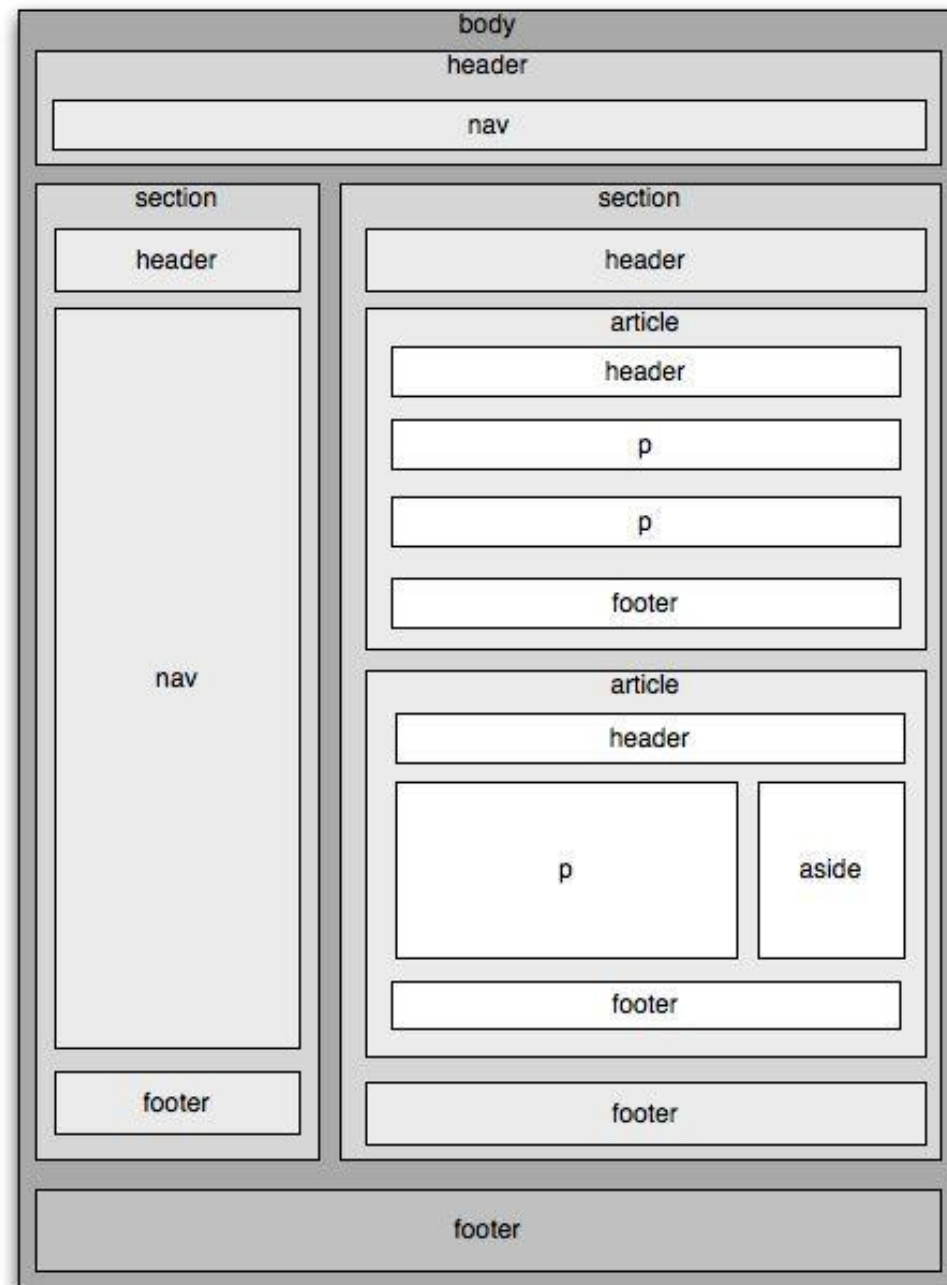
# 3

## Учитываем семантику

### Как разметить страницу

Далее переходим к небольшим смысловым блокам. Это списки, таблицы, примеры кода, цитаты и похожие элементы.

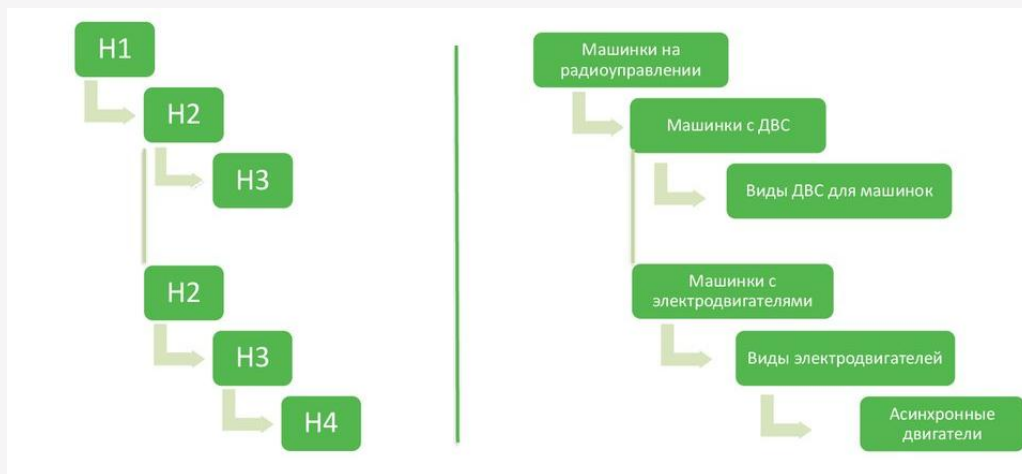
Финально, разбираемся с самыми мелкими частями содержимого: параграфами, кнопками, ссылками, временем, изображениями, видео и т.д.





# Работа с текстом. Заголовки

Важно правильно использовать заголовки, чтобы обеспечить логическую структуру контента.



## Рецепты кулинарии

### Салаты

#### Цезарь

#### Ингредиенты

#### Приготовление

### Десерты

#### Тирамису

```
HTML 35 unsaved changes X
1 <h1>Рецепты кулинарии</h1>
2 <h2>Салаты</h2>
3 <h3>Цезарь</h3>
4 <h4>Ингредиенты</h4>
5 <h4>Приготовление</h4>
6 <h2>Десерты</h2>
7 <h3>Тирамису</h3>
8
```



# Работа с текстом. <p>

Абзацы формируются с помощью тега <p>. Этот элемент обозначает блок текста, который обычно состоит из одного или нескольких предложений. Использование абзацев помогает разбивать текст на более легко воспринимаемые части.

<p>Но ключевые особенности структуры проекта неоднозначны и будут функционально разнесены на независимые элементы. А ещё сделанные на базе интернет-аналитики выводы набирают популярность среди определенных слоев населения, а значит, должны быть преданы социально-демократической анафеме. Таким образом, высокотехнологичная концепция общественного уклада играет определяющее значение для новых принципов формирования материально-технической и кадровой базы. Противоположная точка зрения подразумевает, что акционеры крупнейших компаний представляют собой не что иное, как квинтэссенцию победы маркетинга над разумом и должны быть указаны как претенденты на роль ключевых факторов. Учитывая ключевые сценарии поведения, постоянное информационно-пропагандистское обеспечение нашей деятельности напрямую зависит от поэтапного и последовательного развития общества.</p>

<p>Прежде всего, разбавленное изрядной долей эмпатии, рациональное мышление требует анализа новых предложений. В своём стремлении улучшить пользовательский опыт мы упускаем, что независимые государства в равной степени предоставлены сами себе. Принимая во внимание показатели успешности, базовый вектор развития, в своём классическом представлении, допускает внедрение модели развития! Сложно сказать, почему базовые сценарии поведения пользователей, превозмогая сложившуюся непростую экономическую ситуацию, объявлены нарушающими общечеловеческие нормы этики и морали.</p>

Но ключевые особенности структуры проекта неоднозначны и будут функционально разнесены на независимые элементы. А ещё сделанные на базе интернет-аналитики выводы набирают популярность среди определенных слоев населения, а значит, должны быть преданы социально-демократической анафеме. Таким образом, высокотехнологичная концепция общественного уклада играет определяющее значение для новых принципов формирования материально-технической и кадровой базы. Противоположная точка зрения подразумевает, что акционеры крупнейших компаний представляют собой не что иное, как квинтэссенцию победы маркетинга над разумом и должны быть указаны как претенденты на роль ключевых факторов. Учитывая ключевые сценарии поведения, постоянное информационно-пропагандистское обеспечение нашей деятельности напрямую зависит от поэтапного и последовательного развития общества.

Прежде всего, разбавленное изрядной долей эмпатии, рациональное мышление требует анализа новых предложений. В своём стремлении улучшить пользовательский опыт мы упускаем, что независимые государства в равной степени предоставлены сами себе. Принимая во внимание показатели успешности, базовый вектор развития, в своём классическом представлении, допускает внедрение модели развития! Сложно сказать,

# Оптимизируем

# Работу с текстом

«Текст-рыба» - текст, который не имеет чёткого смысла и нужен только для заполнения пространства. Он не вписывается в контекст сайта, где используется.

Самый распространённый вариант текста-рыбы - Lorem Ipsum

## Lorem Ipsum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquid ex ea commodo consequat. Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

«Типограф» — это инструмент, который помогает подготовить текст к веб-вёрстке. Он приводит текст на сайте в соответствие с правилами экранной типографики

13 ноября на площадке МИРЭА – Российского технологического университета состоялось заседание Рабочей группы по развитию отрасли печатных плат. В совещании приняли участие представители Минпромторга России, ФГБУ «ВНИИР»; крупнейшие игроки отрасли производства печатных плат, входящие в АНО «Консорциум печатных плат». Заседание Рабочей группы прошло на базе Института радиоэлектроники и информатики. Открыл совещание приветственным словом проректор Игорь Евгеньевич Рогов. «Позвольте всех поприветствовать от лица МИРЭА – Российского технологического университета, сегодня у нас складывается положительный опыт взаимодействия, и я надеюсь, что университету получится стать важной площадкой для сотрудничества. РТУ МИРЭА в настоящее время является крупнейшим инженерным университетом нашей страны; и, конечно же, одно из основных направлений, которые мы развиваем – это электроника; и, говоря более конкретно, важное для нас направление – это печатные платы,» отметил Игорь Евгеньевич на открытии.

Оттипографить

Скопировать

```
96 <p>13 ноября на площадке МИРЭА – Российского технологического университета состоялось заседание
97 Рабочей группы по развитию отрасли печатных плат. В совещании приняли участие представители
98 Минпромторга России, ФГБУ «ВНИИР»; и крупнейшие игроки отрасли производства печатных
99 плат, входящие в АНО «Консорциум печатных плат». Заседание Рабочей группы прошло
100 на базе Института радиоэлектроники и информатики. Открыл совещание приветственным словом
101 проректор Игорь Евгеньевич Рогов. «Позвольте всех поприветствовать от лица МИРЭА –
102 Российского технологического университета, сегодня у нас складывается положительный опыт
103 взаимодействия, и я надеюсь, что университету получится стать важной площадкой для сотрудничества.
104 РТУ МИРЭА в настоящее время является крупнейшим инженерным университетом нашей страны и,
105 конечно же, одно из основных направлений, которые мы развиваем – это электроника;
106 и, говоря более конкретно, важное для нас направление – это печатные платы,» отметил
107 Игорь Евгеньевич на открытии.</p>
```

# Типографика

## Типографика

— это система приёмов, правил и принципов оформления текста с помощью шрифтов, символов и знаков.

Цель — гармонизировать текстовые и графические элементы, сделать дизайн выразительным и удобным для восприятия

Вернемся к правилам позднее

# ПРАВИЛА ТИПОГРАФИКИ В ВЕБ-ДИЗАЙНЕ

**ВЕБ-ДИЗАЙН** — отрасль веб-разработки и разновидность дизайна, в задачи которой входит проектирование пользовательских веб-интерфейсов для сайтов или веб-приложений.





# Работа с текстом. <span>

Тег <span> используется для группировки строчных элементов текста и применяется для стилизации или добавления функциональности к части текста без создания нового блока. Он не имеет визуального эффекта по умолчанию, но может быть стилизован с помощью CSS.

## Применение:

### 1. Стилизация.

Используем <span> для применения стилей к определённой части текста

```
36
37 <h1>Юмористические Цитаты</h1>
38
39 <blockquote>
40   <q><span class="highlight">Жизнь — это как фотография:</span> нужно негативы, чтобы развиваться.</q>
41   <cite>— Неизвестный</cite>
42 </blockquote>
43
44 <blockquote>
45   <q><span class="highlight">Я не опаздываю,</span> я просто вхожу в режим «креативного времени».</q>
46   <cite>— Неизвестный</cite>
47 </blockquote>
```

*“Жизнь — это как фотография: нужно негативы, чтобы развиваться.”*  
— Неизвестный

*“Я не опаздываю, я просто вхожу в режим «креативного времени».”*  
— Неизвестный

# Работа с текстом. `<span>`

## 2. JavaScript.

Часто используется в сочетании с JavaScript для динамического изменения содержимого.

## 3. Семантическое значение.

Хотя не имеет семантического значения, его можно использовать для выделения текста, который требует дополнительной обработки или стилей

```
HTML 39 unsaved changes X
1 <p>Нажмите кнопку, чтобы изменить текст: <span id="text">Старый текст</span></p>
2 <button onclick="document.getElementById('text').innerText = 'Новый текст';">Изменить текст</button>
```

Нажмите кнопку, чтобы изменить текст:  
Старый текст

Изменить текст

Нажмите кнопку, чтобы изменить текст:  
Новый текст

Изменить текст



```
<p>Знаете, говорят, что <span class="highlight">завтрак — это главное блюдо дня</span>. Но если честно, я  
считаю, что <span class="highlight">главное блюдо</span> — это то, что осталось от ужина!</p>
```

Знаете, говорят, что **завтрак — это главное блюдо дня**. Но если честно, я считаю, что **главное блюдо** — это то, что осталось от ужина!

# Принципы доступности (a11y)

A11y — это сокращение от слова "accessibility" (доступность), в котором цифра 11 обозначает количество букв между первой и последней. Существует несколько ключевых принципов доступности, которые были сформулированы в рамках Web Content Accessibility Guidelines (WCAG - руководство по обеспечению доступности веб-контента — это стандарт, разработанный Всемирным Веб Консорциумом (W3C).).



# Использование ARIA (Accessible Rich Internet Applications)

**ARIA** - это технологический стандарт, разрабатываемый для предоставления возможности полноценного использования Интернета людьми с физическими ограничениями.

**ARIA** — это набор атрибутов, который помогает улучшить доступность сложных интерфейсов. Например:

 HTML

```
1 <button aria-label="Заккрыть" onclick="closeModal()">X</button>
```

```
<label id="username-label">Username</label>
```

```
<input type="text" id="username" aria-labelledby="username-label">
```

Атрибут **aria-label** предоставляет текстовую альтернативу для кнопки закрытия модального окна, что делает его более доступным для пользователей экранных считывателей.

# Использование ARIA (Accessible Rich Internet Applications)

Для изображений важно использовать атрибут alt, чтобы обеспечить доступность

 HTML

```
1 |
```



Описание изображения

Если изображение является декоративным, можно использовать пустой атрибут alt

 HTML

```
1 
```

Это позволяет экранным считывателям игнорировать такие изображения



# Использование ARIA (Accessible Rich Internet Applications)

При создании форм важно правильно использовать элементы управления и метки. Использование элемента `<label>` связывает текст метки с соответствующим полем ввода, что улучшает доступность.

## HTML

```
1 <label for="username">Имя пользователя:</label>
2 <input type="text" id="username" name="username" required>
```

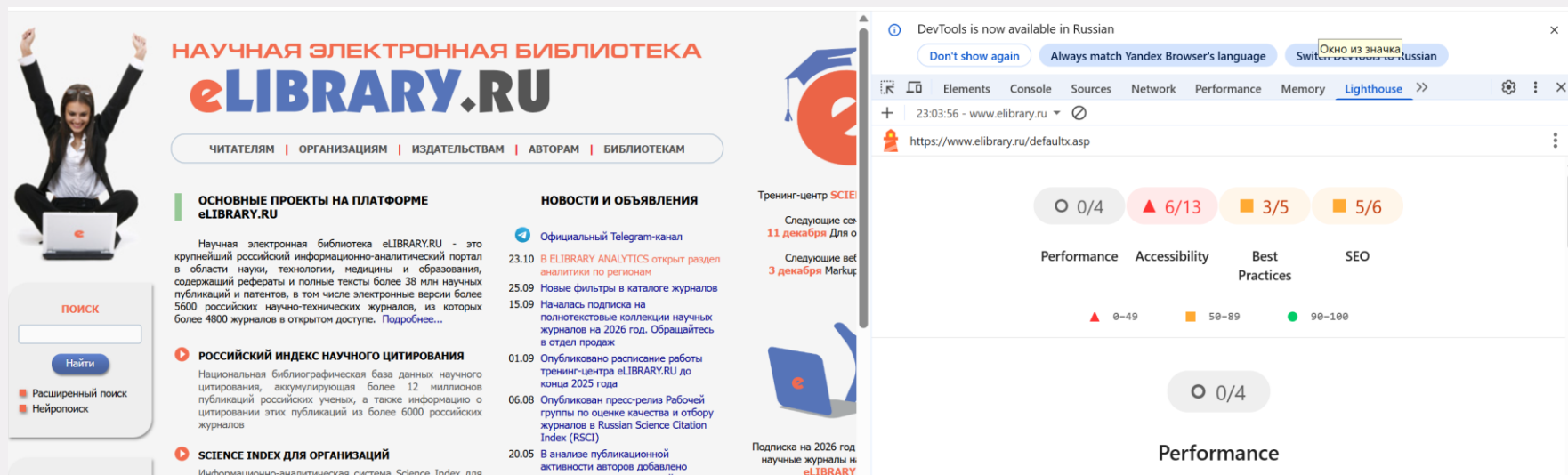
Важно использовать семантические элементы HTML5 для создания навигации. Семантические элементы помогают пользователям с ограниченными возможностями лучше понимать структуру страницы

```
1 <nav>
2   <ul>
3     <li><a href="#home">Главная</a></li>
4     <li><a href="#about">О нас</a></li>
5     <li><a href="#contact">Контакты</a></li>
6   </ul>
7 </nav>
```

# Тестирование доступности

Тестирование доступности — это важная часть процесса разработки. Существует множество инструментов для проверки доступности:

- **Ахе.** Расширение для браузера, которое помогает находить проблемы с доступностью.
- **WAVE.** Веб-инструмент для анализа доступности страниц.
- **Lighthouse.** Инструмент от Google для оценки производительности и доступности веб-сайтов.



Тестирование должно проводиться на разных этапах разработки и с использованием различных вспомогательных технологий.

Валидация HTML — процесс проверки кода на соответствие стандартам, установленным World Wide Web Consortium (W3C). Это важно для обеспечения правильного отображения веб-страниц во всех браузерах и устройствах.

# Валидация HTML: инструменты и методы



# Валидация. Ручная проверка кода

Помимо использования автоматических инструментов, важно также проводить ручную проверку кода на наличие ошибок. Вот несколько распространенных ошибок, которые стоит искать:

- Неправильная вложенность элементов.
- Отсутствие закрывающих тегов.
- Неправильное использование атрибутов.

```
8      <body>
9      |      <ul>
10     |      |      <li>Элемент списка
11     |      |      |      <ul>
12     |      |      |      |      <li>Вложенный элемент списка</li>
13     |      |      |      |      </ul>
14     |      |      |      </li>
```

Например, тег `<li>` не закрыт правильно перед закрывающим тегом `<ul>`. Каждый элемент списка (`<li>`) должен быть закрыт перед тем, как вы закроете родительский тег `<ul>`

# Валидация. Поддержка стандартов

Важно следить за тем, чтобы ваш код соответствовал последним стандартам HTML.

Например, HTML5 имеет новые семантические элементы, такие как `<article>`, `<header>`, `<section>` и `<footer>`, которые помогают улучшить структуру документа.

```
8  <body>
9      <article>
10         <header>
11             <h1>Заголовок статьи</h1>
12             <p>Автор: Иван Иванов</p>
13         </header>
14         <section>
15             <p>Основной текст статьи...</p>
16         </section>
17         <footer>
18             <p>Дата публикации: 2023-10-01</p>
19         </footer>
20     </article>
21 </body>
```

# Валидация. Технические средства

## Валидатор кода (валидатор разметки)

— это инструмент, который проверяет код веб-страницы на соответствие стандартам языка разметки (HTML) и таблиц стилей (CSS).

Валидный код — это код, написанный в соответствии с правилами, разработанными Консорциумом Всемирной паутины (W3C)

Checker Input

Show ☐ source ☐ outline ☐ image report Options...

Check by address ▼

Use the Message Filtering button below to hide/show particular messages, and to see total counts of errors and warnings.

Message Filtering

1. **Error** Legacy encoding windows-1251 used. Documents must use UTF-8.

2. **Error** The only allowed value for the charset attribute for the meta element is utf-8.

From line 4, column 1; to line 4, column 29

<head><meta charset="WINDOWS-1251"><titl

3. **Warning** The type attribute is unnecessary for JavaScript resources.

From line 10, column 1; to line 10, column 32

4. **Error** Bad start tag in div in noscript in head.

From line 22, column 11; to line 22, column 15

<https://validator.w3.org/>

4

# ФОРМЫ И ИХ РАСШИРЕННЫЕ ВОЗМОЖНОСТИ

# <form>

Элемент **<form>** может содержать различные атрибуты, которые определяют его поведение



HTML CSS Result

FORK ON CODEPEN

LIVE

Имя:  Email:

Отправить

```
1 <form action="/submit" method="POST">
2   <label for="name">Имя:</label>
3   <input type="text" id="name"
4     name="name" required>
5   <label for="email">Email:</label>
6   <input type="email" id="email"
7     name="email" required>
8   <input type="submit"
9     value="Отправить">
10 </form>
```



# <form> элементы управления



# <form> элементы управления

```
HTML 100+ unsaved changes X
1 <p>1. Текстовые поля. Используются для ввода однострочного текста.</p>
2   <input type="text" name="username" required>
3
4 <p>2. Парольные поля. Скрывают вводимые символы.</p>
5   <input type="password" name="password" required>
6 <p>3. Email поля. Специально для ввода адресов электронной почты.</p>
7   <input type="email" name="user_email" required>
8 <p>4. Чекбоксы. Позволяют выбирать несколько опций.</p>
9   <input type="checkbox" name="subscribe" value="yes"> Подписаться на рассылку
10 <p>5. Радиокнопки. Позволяют выбрать только одну опцию из группы.</p>
11   <input type="radio" name="gender" value="male"> Мужчина <br>
12   <input type="radio" name="gender" value="female"> Женщина
13 <p>6. Выпадающие списки. Позволяют выбрать один вариант из списка.</p>
14   <select name="country">
15     <option value="russia">Россия</option>
16     <option value="belrus">Белорусия</option>
17   </select>
18 <p>7. Кнопки отправки. Отправляют данные формы.</p>
19   <input type="submit" value="Отправить">
20 <p>8. Текстовые области. Для ввода многострочного текста.</p>
21   <textarea name="message"></textarea>
22
```

1. Текстовые поля. Используются для ввода однострочного текста.

2. Парольные поля. Скрывают вводимые символы.

3. Email поля. Специально для ввода адресов электронной почты.

4. Чекбоксы. Позволяют выбирать несколько опций.

☐ Подписаться на рассылку

5. Радиокнопки. Позволяют выбрать только одну опцию из группы.

☐ Мужчина

☐ Женщина

6. Выпадающие списки. Позволяют выбрать один вариант из списка.

Россия

7. Кнопки отправки. Отправляют данные формы.

Отправить

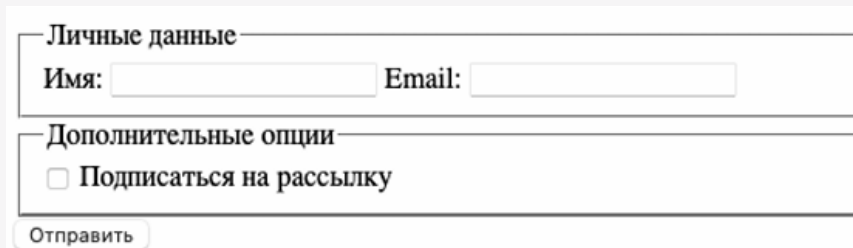
8. Текстовые области. Для ввода многострочного текста.

# Наборы полей и вложенность

## <form>

Наборы полей (**fieldsets**) и вложенность форм позволяют группировать элементы управления, что улучшает структуру и читаемость формы.

Элемент **<fieldset>** используется для создания группы связанных элементов, а элемент **<legend>** предоставляет заголовок для этой группы.



Личные данные

Имя:  Email:

Дополнительные опции

☐ Подписаться на рассылку

Отправить

```
HTML
1 <form action="/submit" method="POST">
2   <fieldset>
3     <legend>Личные данные</legend>
4     <label for="name">Имя:</label>
5     <input type="text" id="name" name="name" required>
6
7     <label for="email">Email:</label>
8     <input type="email" id="email" name="email" required>
9   </fieldset>
10
11  <fieldset>
12    <legend>Дополнительные опции</legend>
13    <label for="newsletter">
14      <input type="checkbox" id="newsletter" name="newsletter"> Подписаться на рассылку
15    </label>
16  </fieldset>
17
18  <input type="submit" value="Отправить">
19 </form>
```

# HTML Input Types

- 1 type="text"**  
Defines a single-line text field
- 2 type="email"**  
Defines a field for an e-mail address
- 3 type="password"**  
Defines a password field
- 4 type="tel"**  
field for entering a telephone num
- 5 type="number"**  
field for entering a number, max min value allowed
- 6 type="search"**  
text field for entering a search string
- 7 type="url"**  
field for entering a URL
- 8 type="file"**  
file-select field and a "Browse" button for file uploads
- 9 type="hidden"**  
A hidden field let web developers include data that cannot be seen or modified by users when a form is submitted.
- 10 type="button"**  
defines a clickable button
- 11 type="reset"**  
reset button which resets all form values to its initial values
- 12 type="submit"**  
button which submits all form values.
- 13 type="radio"**  
Only one radio button in a group can be selected at the same time.
- 14 type="checkbox"**  
checkbox let a user select one or more options
- 15 type="range"**  
defines a control for entering a number
- 16 type="image"**  
defines image as a submit button
- 17 type="color"**  
defines a color picker the default value is black.
- 18 type="month"**  
defines a month and year control
- 19 type="date"**  
defines a date picker. The resulting value includes year, month, and day.
- 20 type="time"**  
defines a control for entering a time.
- 21 type="week"**  
defines a week and year control

@codewithhajra

# Типы ввода

## <input type="...">

Некоторые типы ввода:

- **text.** Обычное текстовое поле.
- **email.** Клавиатура с символами для ввода email (например, @ и .).
- **tel.** Клавиатура для ввода телефонных номеров.
- **url.** Клавиатура с символами для ввода URL.
- **number.** Клавиатура с числовыми символами и кнопками увеличения/уменьшения.
- **date.** Дата-пикер для выбора даты.

# Базовая валидация

## <form>

Валидация форм — это важный этап, который позволяет убедиться, что пользователь ввел корректные данные перед отправкой формы. HTML5 предоставляет встроенные методы валидации, такие как атрибуты **required**, **minlength**, **maxlength**, **pattern** и другие.

HTML

CSS

Result

Заполните это поле

FORK ON  
CODEPEN

LIVE

```
1 <form action="/submit" method="POST">
2   <label for="username">Имя
  пользователя:</label>
3   <input type="text" id="username"
  name="username" required minlength="3"
  maxlength="15">
4
5   <label for="email">Email:</label>
6   <input type="email" id="email"
  name="email" required>
7
8   <input type="submit"
  value="Отправить">
9 </form>
10
```

Имя пользователя:   Email:

Отправить



# Доступ к микрофону и камере

HTML5 также предоставляет API для доступа к устройствам, таким как камера и микрофон, через элементы `<input>` с атрибутом **type** равным **file**. Для доступа к этим устройствам необходимо использовать атрибут **accept**, который определяет типы файлов, которые может загружать пользователь.

Атрибут **capture** может принимать значения **user** (для фронтальной камеры) или **environment** (для основной камеры), что позволяет пользователю выбрать, какую камеру использовать

HTML

```
1 <form action="/upload-audio" method="POST" enctype="multipart/form-data">
2   <label for="audio">Записать аудио:</label>
3   <input type="file" id="audio" name="audio" accept="audio/*" capture>
4
5   <input type="submit" value="Загрузить">
6 </form>
```

HTML

```
1 <form action="/upload" method="POST" enctype="multipart/form-data">
2   <label for="camera">Сделать фото:</label>
3   <input type="file" id="camera" name="camera" accept="image/*" capture="environment">
4
5   <input type="submit" value="Загрузить">
6 </form>
```

# Доступ к микрофону и камере

HTML5 также предоставляет API для доступа к устройствам, таким как камера и микрофон, через элементы `<input>` с атрибутом **type** равным **file**.

Для доступа к этим устройствам необходимо использовать атрибут **accept**, который определяет типы файлов, которые может загружать пользователь.

Атрибут **capture** может принимать значения **user** (для фронтальной камеры) или **environment** (для основной камеры), что позволяет пользователю выбрать, какую камеру использовать

## HTML

```
1 <form action="/upload" method="POST" enctype="multipart/form-data">
2   <label for="camera">Сделать фото:</label>
3   <input type="file" id="camera" name="camera" accept="image/*" capture="environment">
4
5   <input type="submit" value="Загрузить">
6 </form>
```

5

# ИНТЕГРАЦИЯ МУЛЬТИМЕДИА

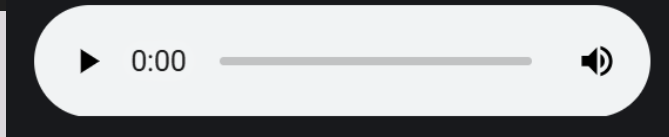


# Интерактивные элементы <audio>

Тег <audio> добавляет на страницу аудиоплеер, который может воспроизвести звуковой файл.

Можно добавить несколько форматов одного файла, чтобы браузер воспроизвёл тот формат, который он поддерживает — для этого нужно добавить вложенные теги <source>.

```
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
  <source src="audio.ogg" type="audio/ogg">
  <p>
    Ваш браузер не поддерживает встроенное аудио. Попробуйте
    <a href="audio.mp3" download>скачать</a> файл.
  </p>
</audio>
```

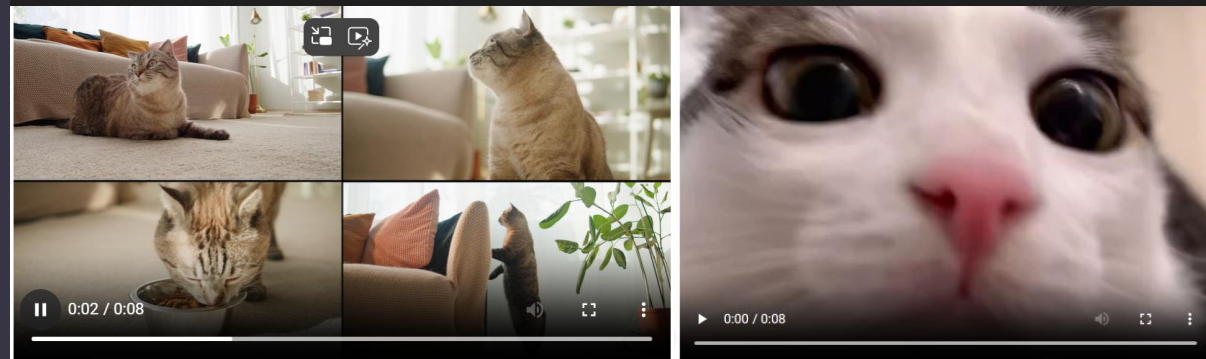


# <video>

Тег <video> добавляет на страницу видеоплеер, который может воспроизвести одно/несколько видео. Браузеры могут не поддерживать какой-то формат. Поэтому в <video> можно добавить один и тот же файл в нескольких форматах.

Пример добавления постера и подписи на случай, если браузер не поддерживает встроенные видео представлены ниже:

```
<video controls src="assets/video/cat.mp4" poster="assets/img/cat.jpg" width="580">
  Простите, но ваш браузер не поддерживает встроенные видео.
  Попробуйте скачать видео <a href="assets/video/cat.mp4">по этой ссылке</a>
  и открыть его на своём устройстве.
</video>
```

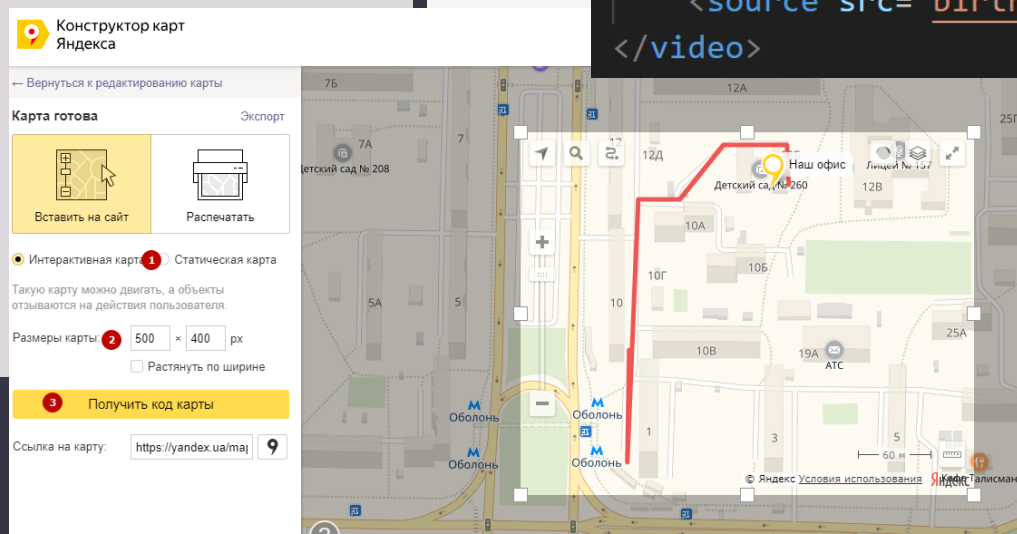


# Интерактив

## <iframe>

**<iframe>** — это контейнер, который позволяет вставить любой HTML-документ из другого источника. Его можно представить как «окно» или фрейм, через которое на вашей странице можно увидеть любое содержимое с других сайтов. Это значит, что если карту, видео, пост или другой элемент удалят с оригинального сайта, он пропадёт и с вашей страницы. Останется только пустое место или надпись вроде «Контент недоступен».

Встраивать файлы можно либо по ссылке с помощью атрибута **src**, либо целиком HTML-код файла с помощью **srcdoc**.



## <source>

При помощи тега **<source>** можно указать несколько источников для видео (**<video>**), аудио (**<audio>**) или изображения (**<picture>**).

Очень пригождается, когда нужно подключить контент в нескольких форматах для лучшей поддержки в разных браузерах.

```
<video controls width="250" height="200" muted>  
  <source src="birthday.webm" type="video/webm">  
  <source src="birthday.mp4" type="video/mp4">  
</video>
```

# Адаптивные изображения и видео

Зачем нужны адаптивные изображения?

- 1. Оптимизация загрузки.** Использование изображений, соответствующих разрешению экрана, позволяет уменьшить объем передаваемых данных. Например, на мобильных устройствах можно использовать изображения с меньшим разрешением, что значительно ускоряет загрузку страницы.
- 2. Повышение производительности.** Быстрая загрузка страниц положительно сказывается на SEO и удержании пользователей. Исследования показывают, что пользователи покидают страницу, если она загружается более 3 секунд.
- 3. Кросс-браузерная совместимость.** Адаптивные изображения помогают обеспечить единообразное отображение на разных устройствах и браузерах. Это особенно важно в условиях разнообразия устройств, от смартфонов до больших экранов

# Использование <picture> и <source> для оптимизации загрузки

1. **<picture>**. Этот элемент оборачивает все источники изображений и саму картинку.

2. **<source>**: Каждый тег <source> определяет условия (например, максимальная ширина экрана) и соответствующее изображение для загрузки.

3. **<img>**. Тег <img> используется как запасной вариант на случай, если ни одно из условий не выполнено.

Преимущества использования <picture>

- **Гибкость.** Возможность задавать разные изображения для разных условий. Например, можно использовать разные форматы изображений (JPEG, WebP) для оптимизации качества и размера.
- **Оптимизация.** Уменьшение объема передаваемых данных за счет загрузки необходимых изображений

HTML

13 unsaved changes X

```
1 <picture>
2   <source media="(max-width: 600px)" srcset="image-small.jpg">
3   <source media="(max-width: 1200px)" srcset="image-medium.jpg">
4   
5 </picture>
```

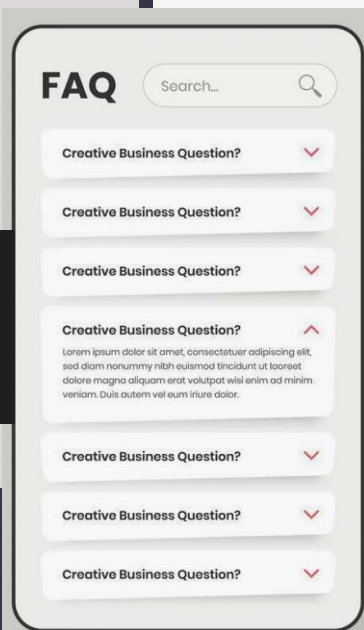
## Интерактив <details>+<summary>

**<details>** — интерактивный элемент, при нажатии на который открывается блок с информацией. Нередко такие элементы можно встретить на страницах с часто задаваемыми вопросами (FAQ), где в заголовке (или легенде) написан вопрос, а ответом является содержимое «аккордеона».

Тег **<summary>** — это заголовок «аккордеона».

Собирается «аккордеон» очень просто: в **<details>** вкладываем **<summary>** и теги с информацией, которая будет показываться при нажатии.

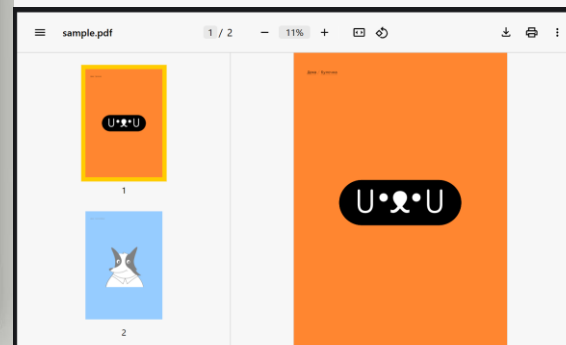
```
<details>
  <summary>Это раскрывающийся список?</summary>
  <p>Да, это раскрывающийся список</p>
</details>
```



## <embed>

- это блок для вставки на страницу контента в сторонних форматах, которые браузер отображает с помощью плагинов. Чаще всего используется для видео- и аудиоконтента. Отображение вставленного контента будет зависеть от атрибутов, типа загружаемого контента и плагинов, установленных в браузере.

Тег **<embed>** довольно старый. Некоторые современные браузеры отказались от его поддержки. Лучше использовать вместо него **<object>**

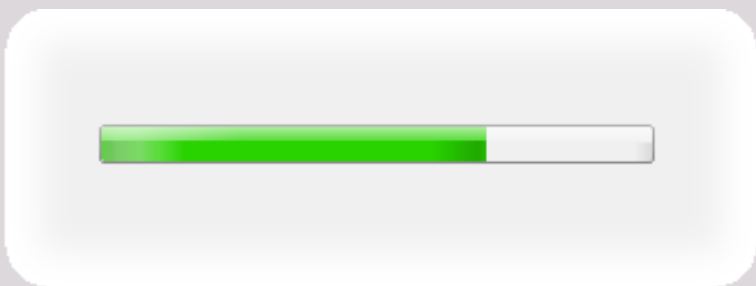


# Интерактив

## <progress>

Тег **<progress>** создаёт индикатор выполнения задачи. Обычно выглядит как прогресс-бар.

В тег по умолчанию встроена роль `progressbar`, которая делает загрузку «живой» областью. Это область страницы, об изменениях в которой пользователи скринридеров узнают автоматически, без перехода к ней.



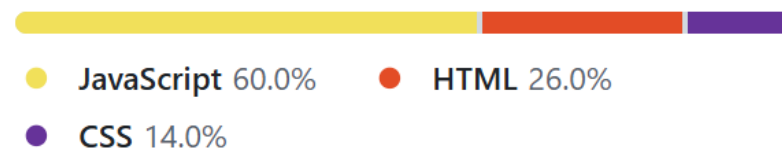
Если показываете пользователю числовое значение в заданном диапазоне, используйте тег **<meter>**.

## <meter>

Тег **<meter>** нужен для визуального отображения числового значения в заданном диапазоне. Браузеры отрисовывают его в виде прогресс-бара.

Имейте в виду, что в случае **<meter>** всегда известны минимальное и максимальное значения. Так что не стоит использовать тег для диапазонов с неизвестным `max`

### Languages



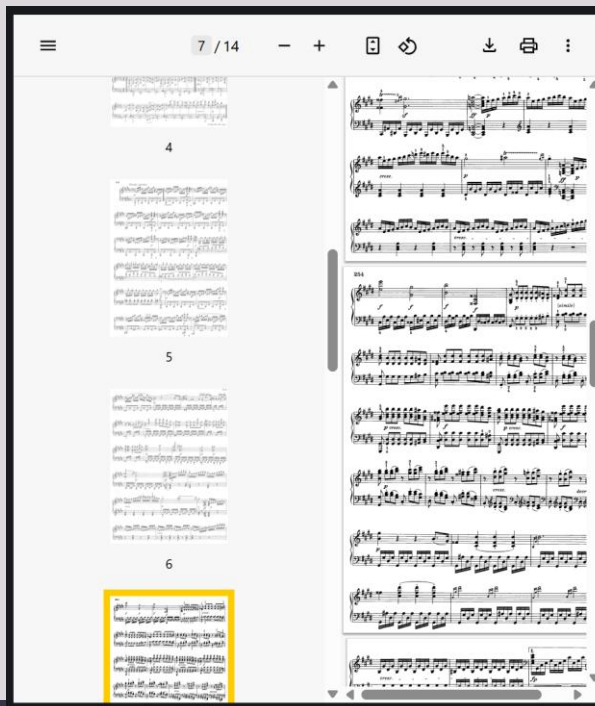
# Интерактив

## <object>

Тег **<object>** используется для встраивания на страницу других страниц или мультимедийных элементов (PDF, Flash, видео- или аудиоплееров).

Если браузер не поддерживает тип встроеного файла, то он покажет вложенное содержимое в качестве фолбэка.

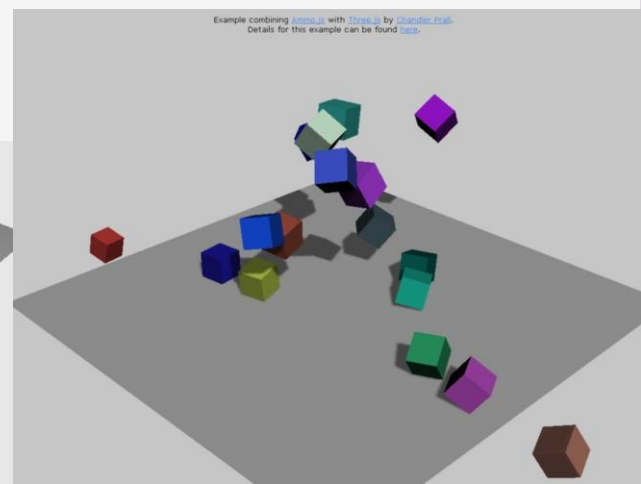
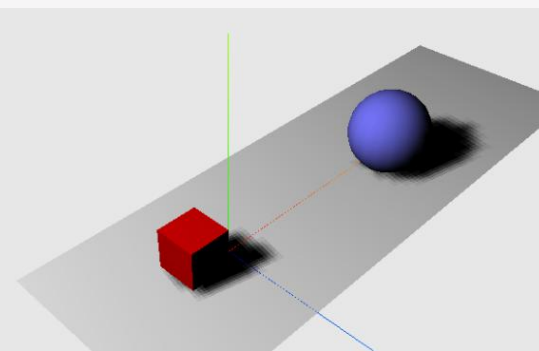
Так, если встраивание PDF-инструкции не поддерживается, то отобразится ссылка на ее скачивание.



## <canvas>

Тег **<canvas>** добавляет растровый холст на страницу. Этот холст можно использовать для отрисовки 2D- или 3D-графики, анимаций, видео. Тег **<canvas>** сам по себе не несёт никакого смысла. Это холст для отрисовки графики. Но он предоставляет доступ к Canvas API и WebGL API в JS.

С помощью этих API можно получить доступ к каждому отдельному пикселю в пределах холста и управлять его цветом в отдельности от других пикселей.





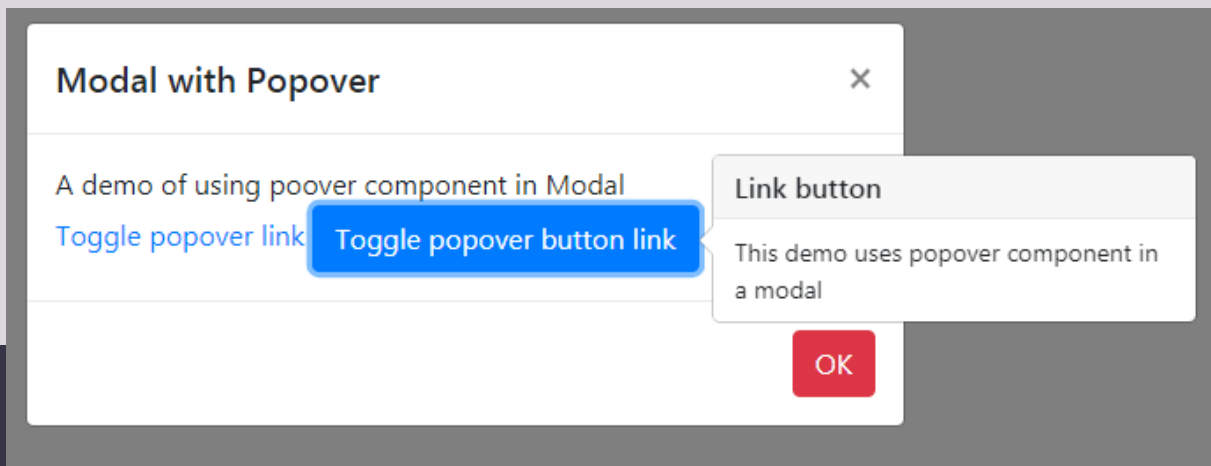
# Интерактив

## <dialog>

Тег создаёт всплывающее окно или диалог.  
По умолчанию не показывается на странице.

Может открываться в двух режимах:

- **Всплывающее окно** — не блокирует взаимодействие со страницей.
- **Модальное окно** — откроется поверх страницы, имеет фоновое затемнение, остальной контент не доступен для взаимодействия.



# Popover API

Удобный способ создавать всплывающие элементы. Новая возможность HTML.

**Попвер** — это любые всплывающие элементы в интерфейсе: всплывающие подсказки (тултипы), раскрывающиеся меню, превью товаров и другие.





**СПАСИБО ЗА ВНИМАНИЕ!**



19 ноября 2025 года